Priority Queues / Binary Heaps

The Priority Queue ADT

- A collection Q of comparable elements that supports the following operations:
  - `insert(x)`: add an element to Q (compare to enqueue)
  - `deleteMin()`: return the minimum element in Q and delete it from Q (compare to dequeue).

Implementing Priority Queues

Implementing Priority Queues

- Idea 1: Use a list.
  `insert(x)`: $O(1)$, `deleteMin()`: $O(N)$

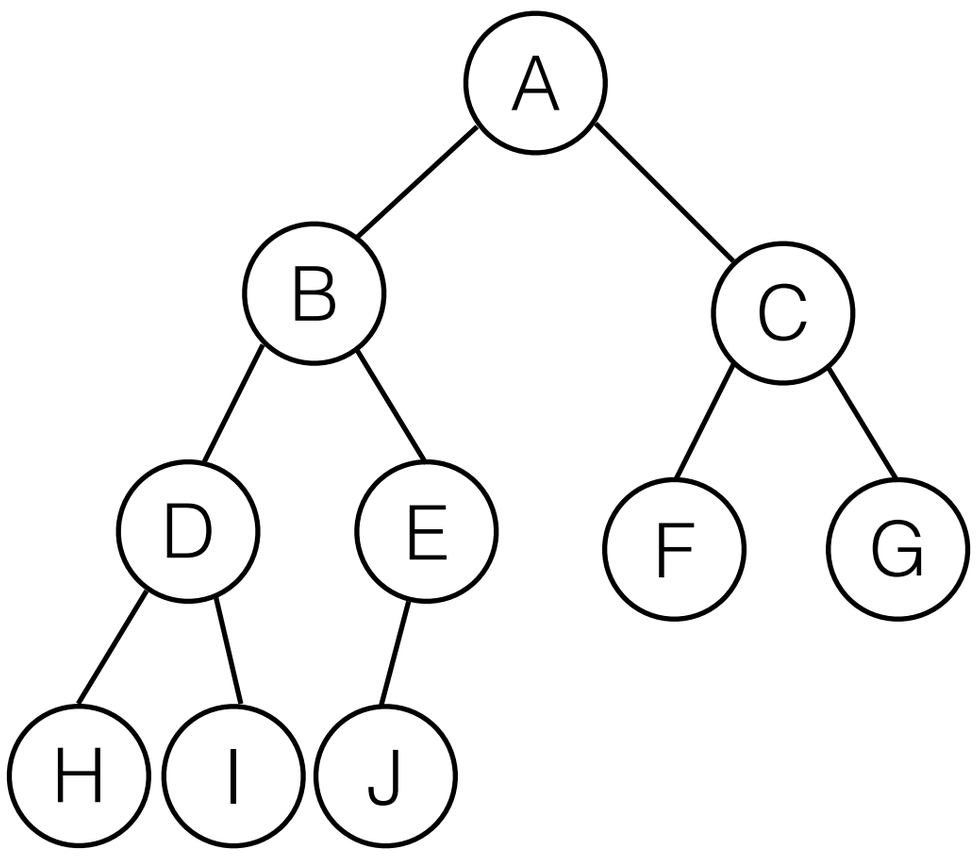
Implementing Priority Queues

- Idea 1: Use a list.
  `insert(x)`: $O(1)$, `deleteMin()`: $O(N)$

- Can do better with a **heap** data structure
  - getMin() in $O(1)$
  - Inserting N items in $O(N)$
  - This gives a sorting algorithm in $O(N \log N)$

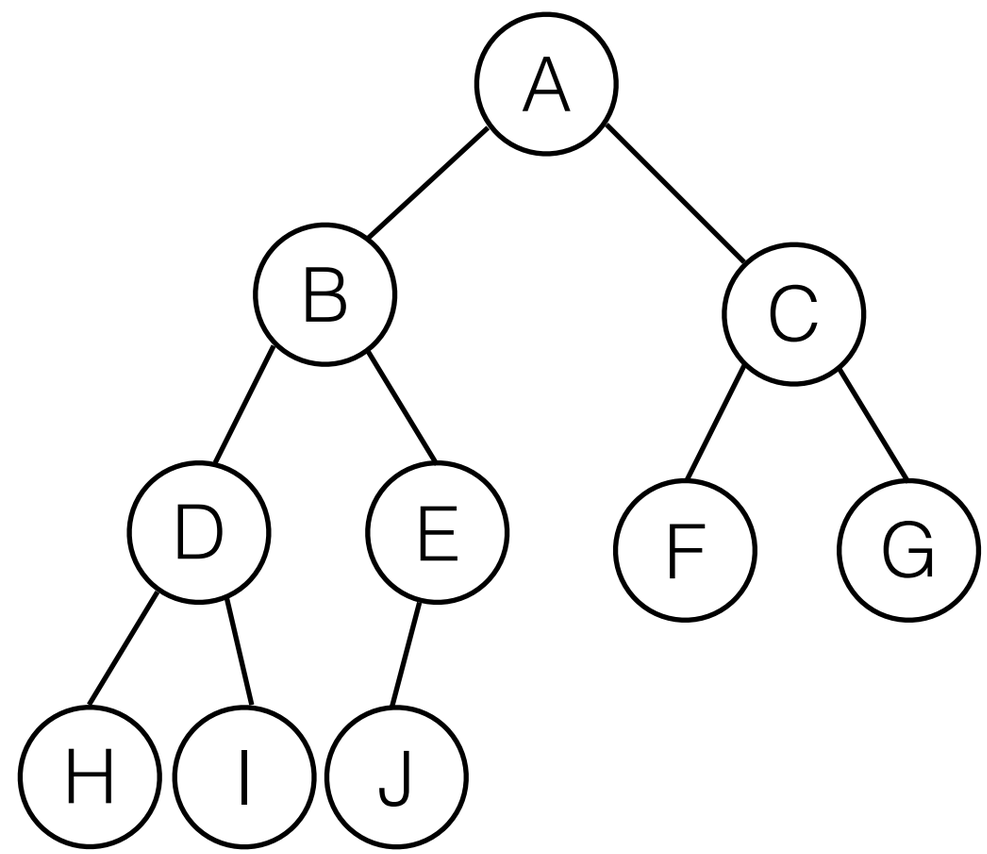
Review: Complete Binary Trees

- All levels are completely full (except possibly the last).



Storing Complete Binary Trees in Arrays

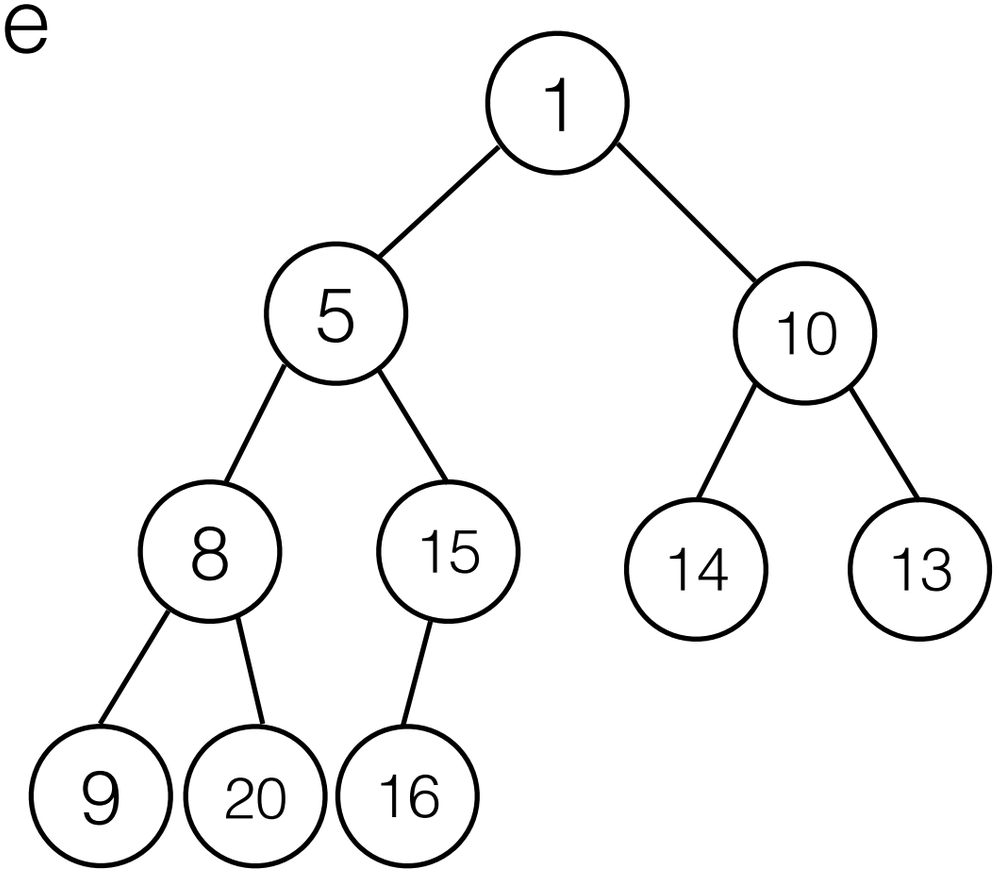
- The shape of a complete binary tree with N nodes is unique.
- We can store such trees in an array in level order.
- Traversal is easy
  - leftChild(i) = 2i
  - rightChild(i) = 2i +1
  - parent(i) = $\lfloor i/2 \rfloor$



	A	B	C	D	E	F	G	H	I	J		

Heap

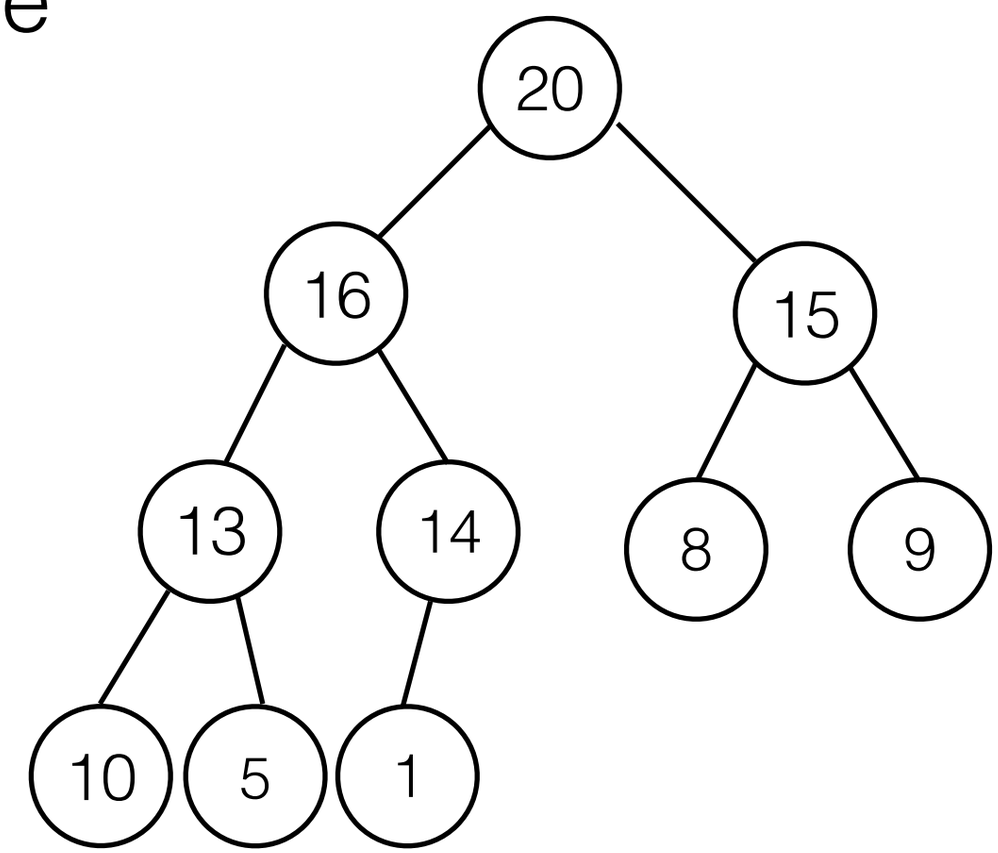
- A heap is a complete binary tree stored in an array, with the following **heap order property**
  - For every node *n* with value x:
    - The values of all nodes in the subtree rooted in *n* are greater or equal than x.



	1	5	10	8	15	14	13	9	20	16			

Max-Heap

- A heap is a complete binary tree stored in an array, with the following **heap order property**:
  - For every node *n* with value x:
    - The values of all nodes in the subtree rooted in *n* are **less or equal** than x.

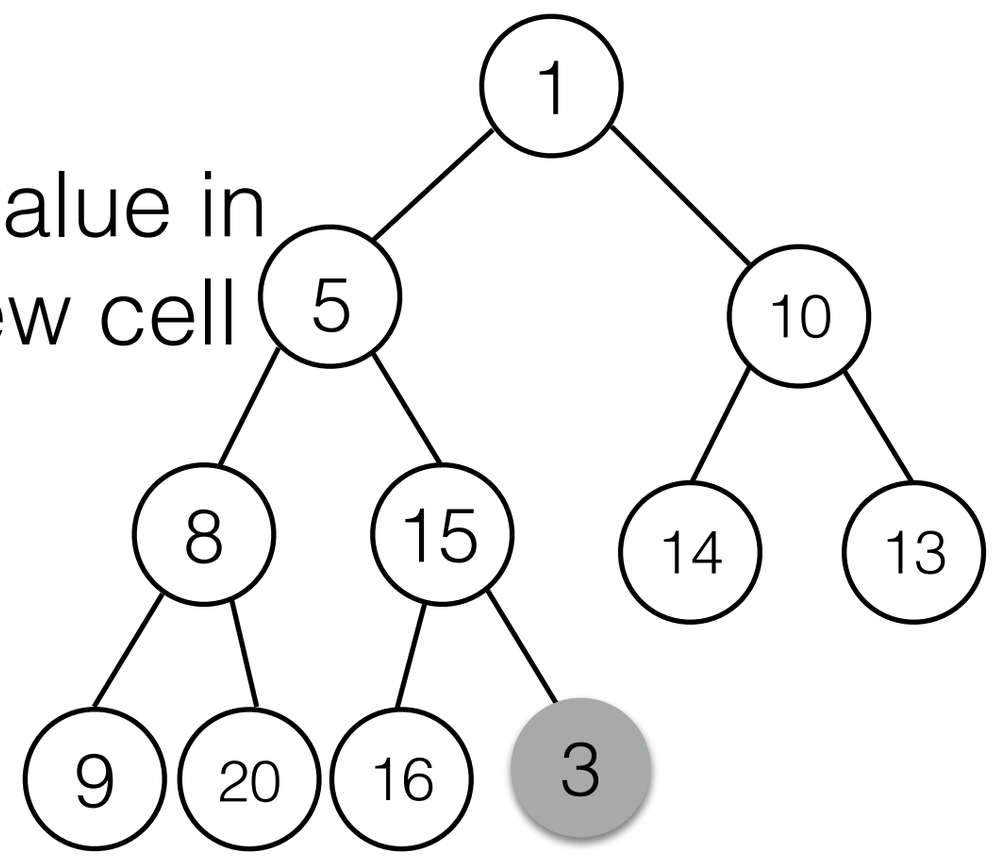


	20	16	15	13	14	8	9	10	5	1		

Min-Heap - `insert(x)`

- Attempt to insert at last array position (next possible leaf in the last layer).
- If heap order property is violated, ***percolate*** the value ***up***
  - Swap that value ("hole") and value in the parent cell, then try the new cell
  - If heap order is still violated, continue until correct position is found

insert(3)

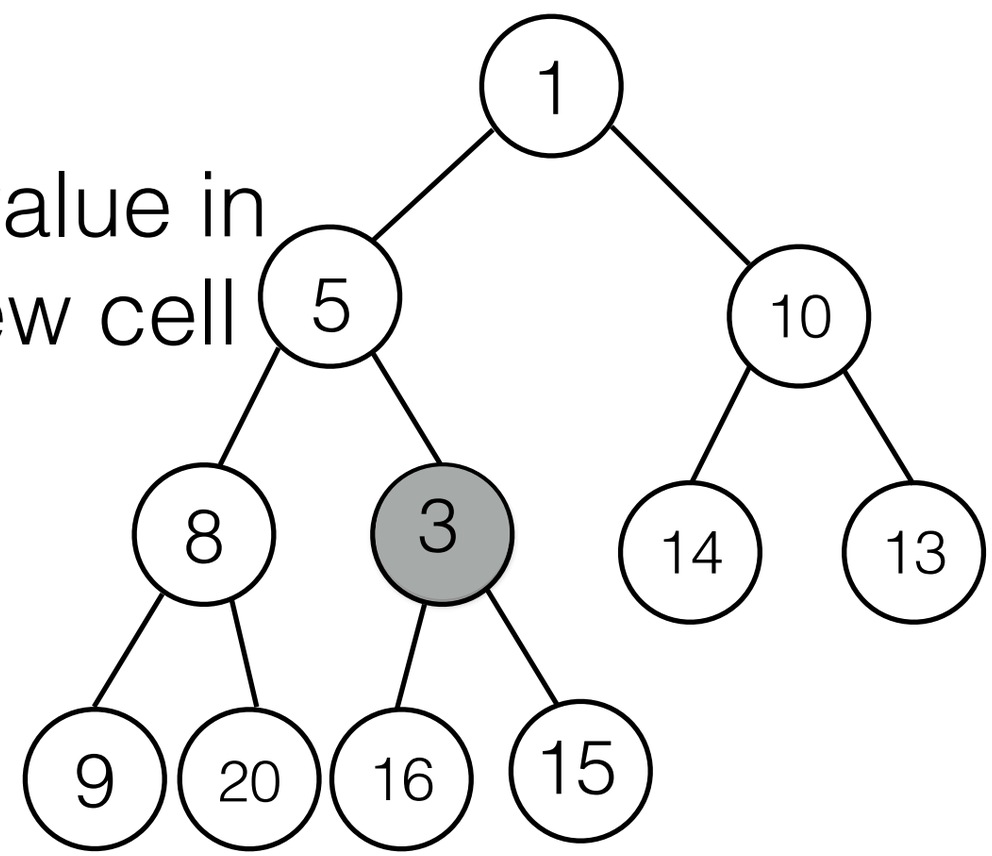


	1	5	10	8	15	14	13	9	20	16	3		

Min-Heap - `insert(x)`

- Attempt to insert at last array position (next possible leaf in the last layer).
- If heap order property is violated, ***percolate*** the value ***up***
  - Swap that value ("hole") and value in the parent cell, then try the new cell
  - If heap order is still violated, continue until correct position is found

insert(3)

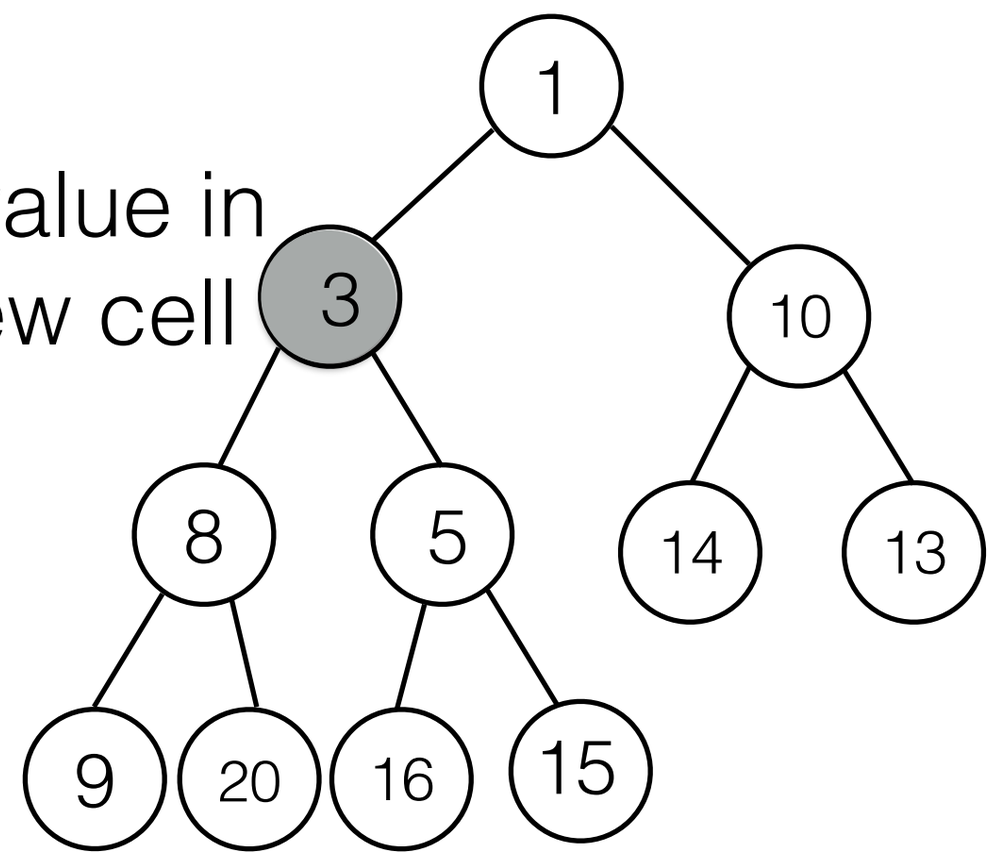


	1	5	10	8	3	14	13	9	20	16	15		

Min-Heap - `insert(x)`

- Attempt to insert at last array position (next possible leaf in the last layer).
- If heap order property is violated, ***percolate*** the value ***up***
  - Swap that value ("hole") and value in the parent cell, then try the new cell
  - If heap order is still violated, continue until correct position is found

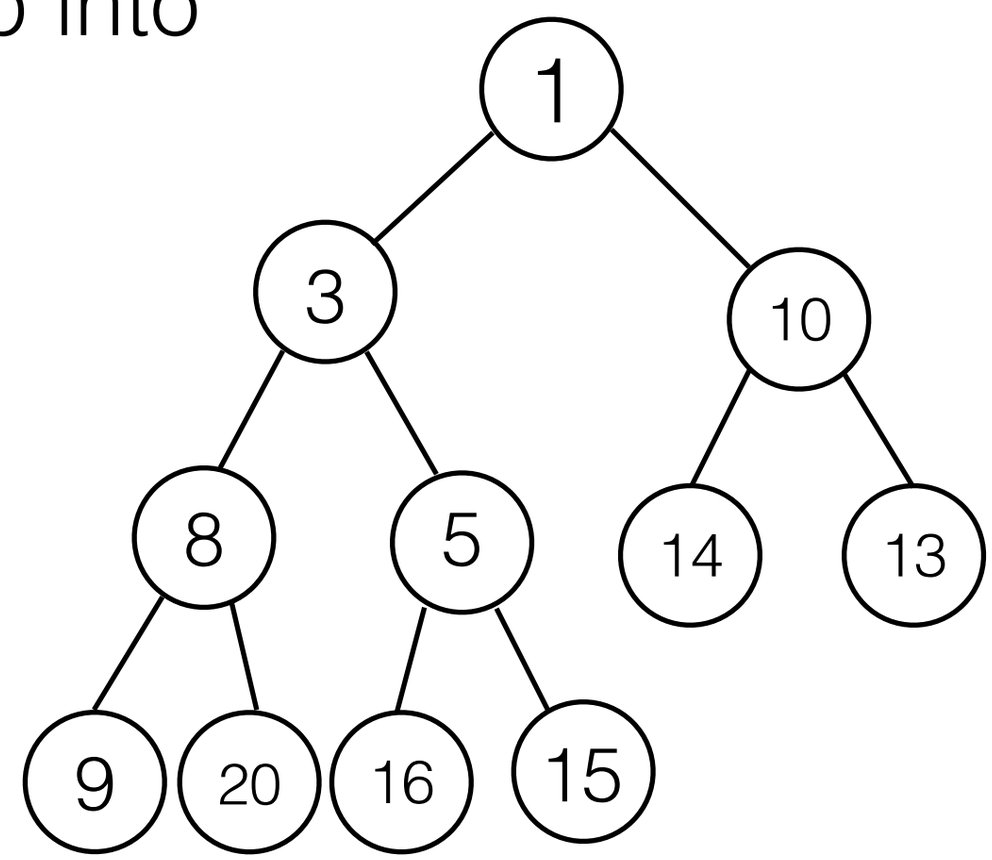
insert(3)



	1	3	10	8	5	14	13	9	20	16	15		

Min-Heap - `deleteMin()`

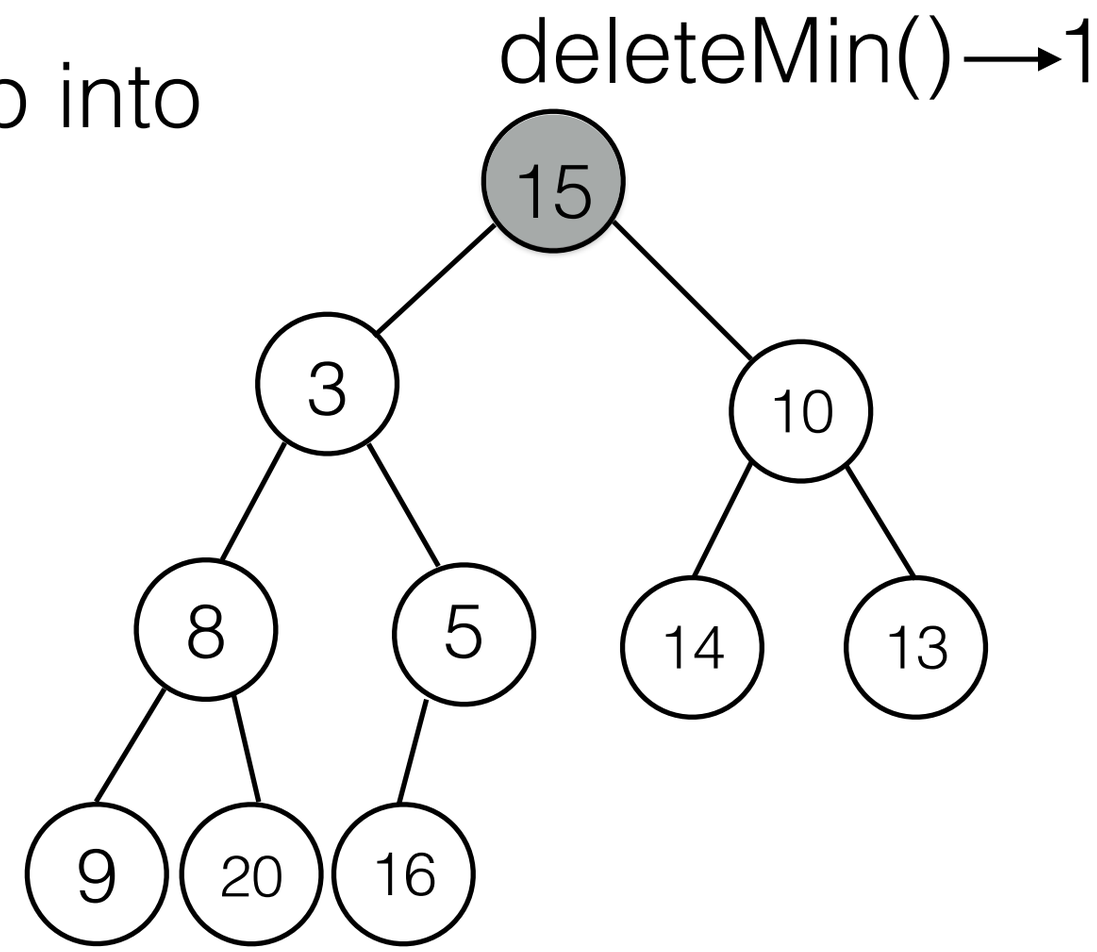
- The minimum is always at the root of the tree
- Remove lowest item, creating an empty cell in the root
- Try to place last item in the heap into the root
  - If heap order is violated, ***percolate*** the value ***down***
    - Swap with the smaller child until the correct position is found



	1	3	10	8	5	14	13	9	20	16	15		

Min-Heap - `deleteMin()`

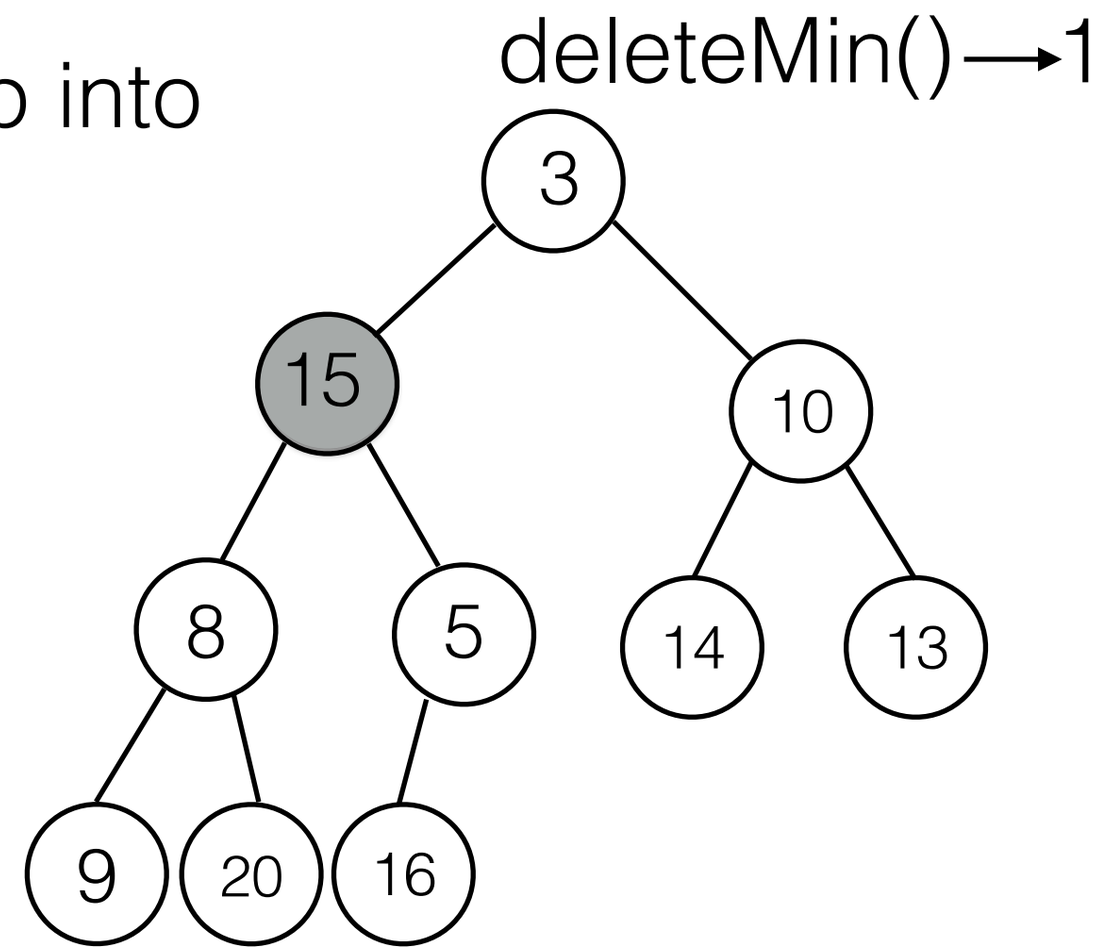
- The minimum is always at the root of the tree
- Remove lowest item, creating an empty cell in the root
- Try to place last item in the heap into the root
  - If heap order is violated, ***percolate*** the value ***down***
    - Swap with the smaller child until the correct position is found



	15	3	10	8	5	14	13	9	20	16			

Min-Heap - `deleteMin()`

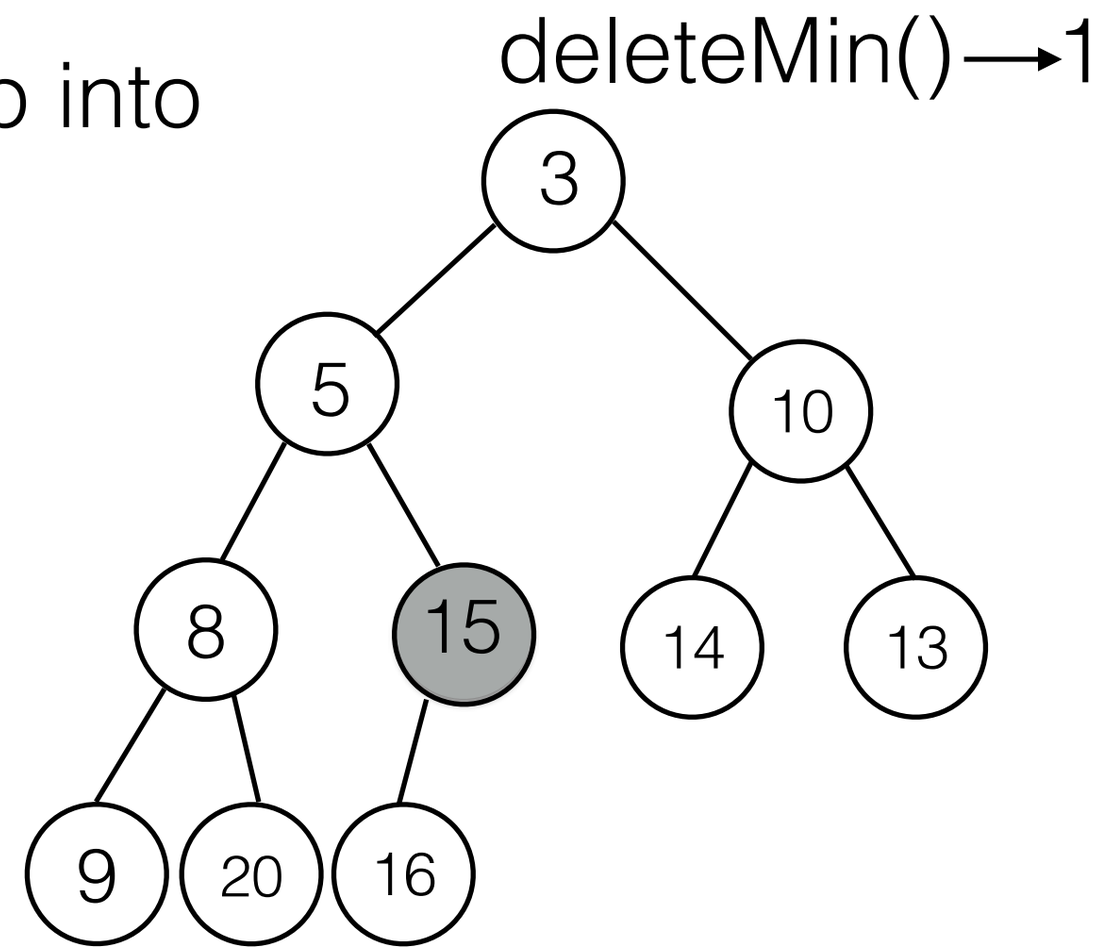
- The minimum is always at the root of the tree
- Remove lowest item, creating an empty cell in the root
- Try to place last item in the heap into the root
  - If heap order is violated, ***percolate*** the value ***down***
    - Swap with the smaller child until the correct position is found



	3	15	10	8	5	14	13	9	20	16			

Min-Heap - `deleteMin()`

- The minimum is always at the root of the tree
- Remove lowest item, creating an empty cell in the root
- Try to place last item in the heap into the root
  - If heap order is violated, ***percolate*** the value ***down***
    - Swap with the smaller child until the correct position is found



	3	5	10	8	15	14	13	9	20	16			

Running Time for Binary Heap Operations

- Because a heap is a *complete binary tree*, it's height is about $\log \mathrm{N}$ (see the segment on structural induction).

- Worst-case running time for `insert(x)` and `deleteMin()` is therefore $\mathrm{O}(\log \mathrm{N})$.

- getMin() is $\mathrm{O}(1)$.

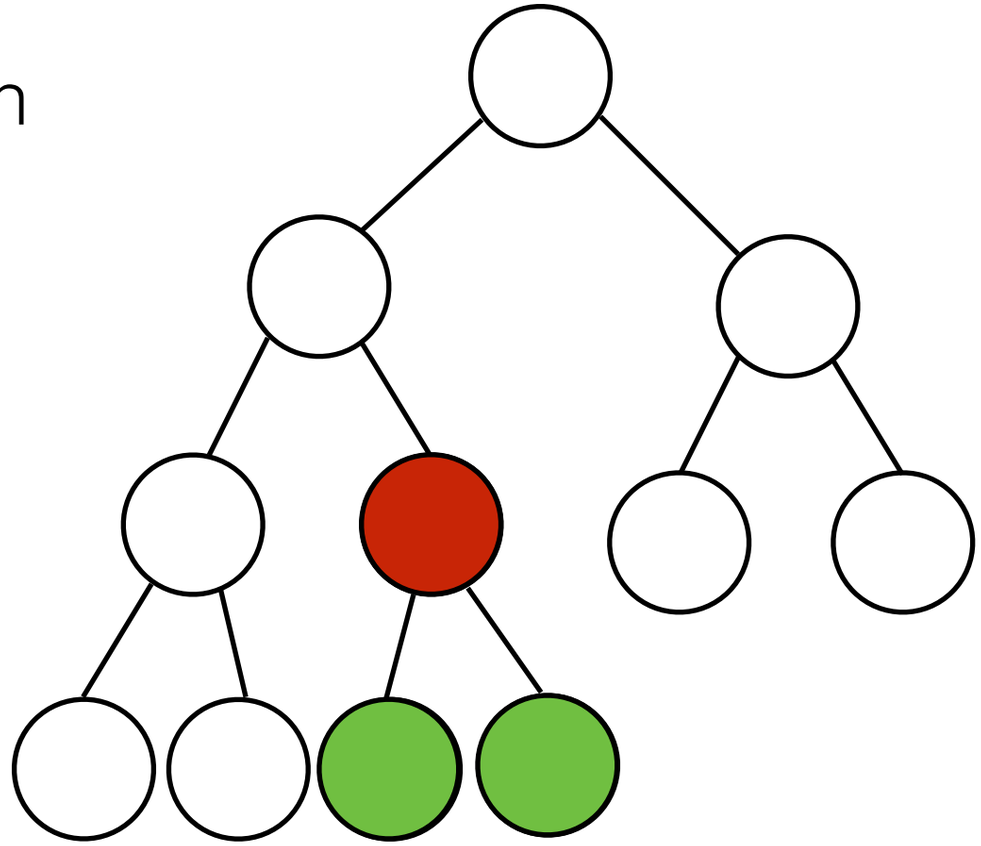
- There is a simple in-place "`heapify`" algorithm that converts an unordered array into a heap in `O(N)` time.

Building a Heap

- Want to convert a collection of $\mathbf{N}$ items into a heap.
- Each `insert(x)` takes $\mathbf{O(\log N)}$ time in the worst case, so the total time is $\mathbf{O(N \log N)}$.
- Can do this in-place and show a better bound of $\mathbf{O(N)}$. This operation is called ***`heapify`.***

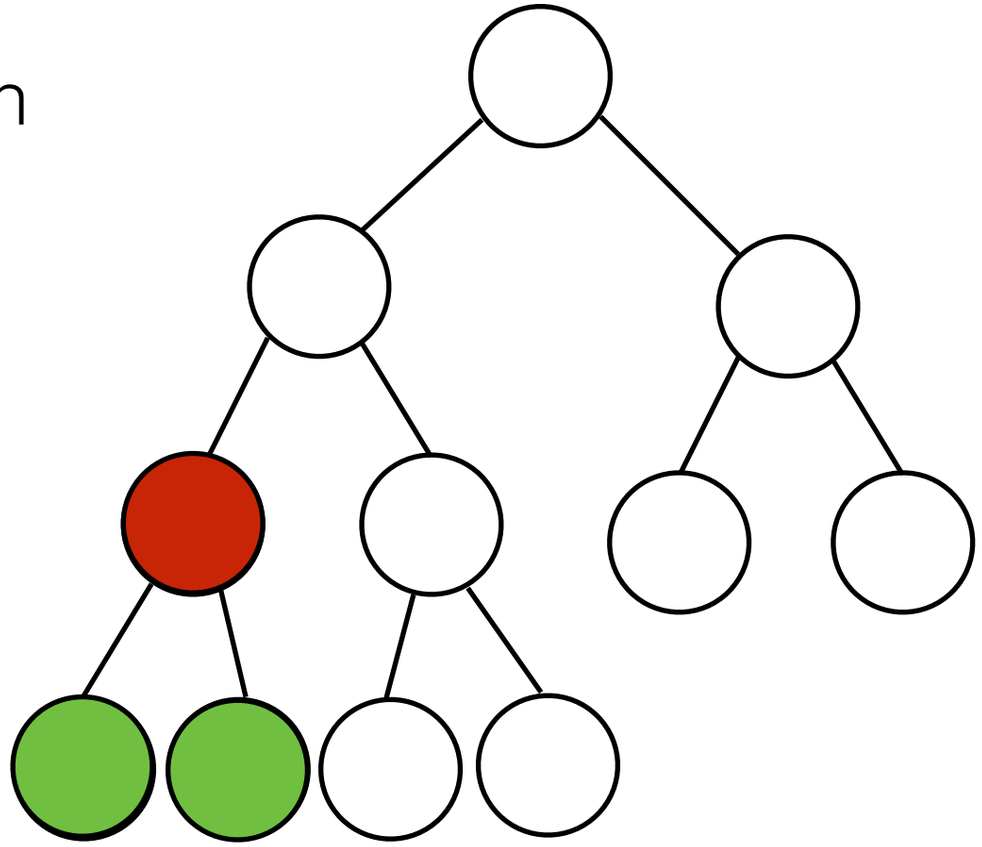
Heapify

- Start with an unordered array.
- `percolateDown(i)` assumes that both subtrees under *i* are already heaps.
- Idea: restore heap property bottom-up.
  - Make sure all subtrees in the two last layers are heaps.
  - Then move up layer-by-layer.



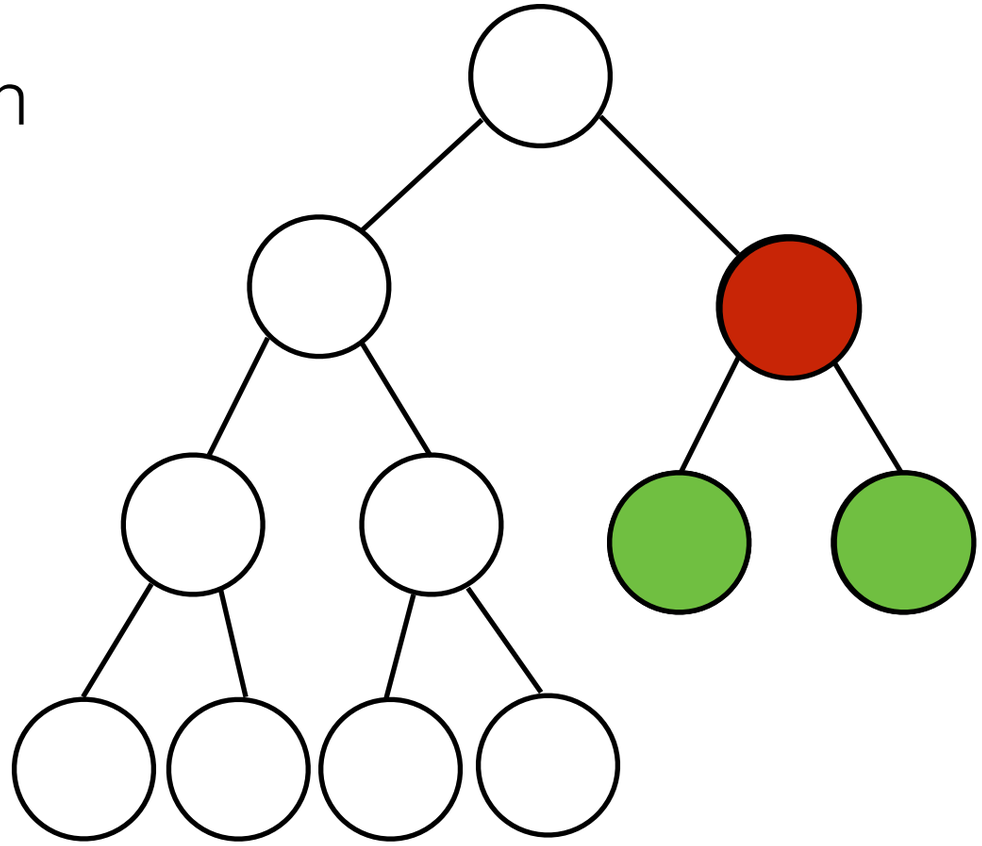
Heapify

- Start with an unordered array.
- `percolateDown(i)` assumes that both subtrees under *i* are already heaps.
- Idea: restore heap property bottom-up.
  - Make sure all subtrees in the two last layers are heaps.
  - Then move up layer-by-layer.



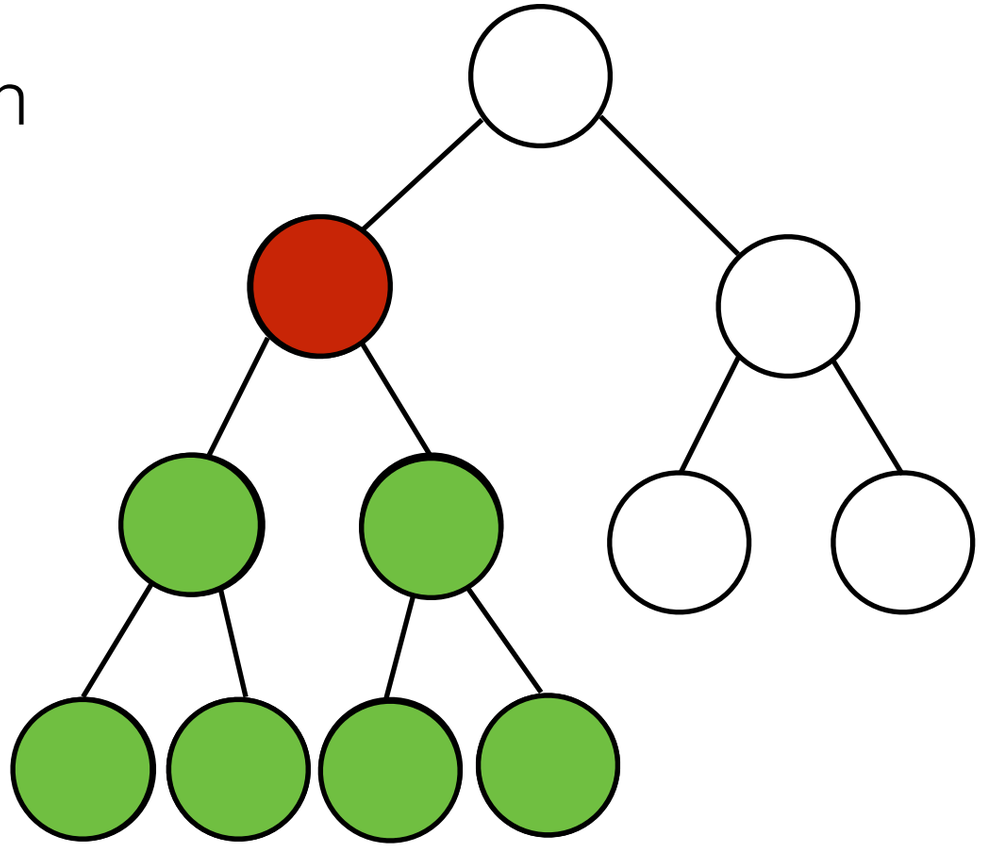
Heapify

- Start with an unordered array.
- `percolateDown(i)` assumes that both subtrees under *i* are already heaps.
- Idea: restore heap property bottom-up.
  - Make sure all subtrees in the two last layers are heaps.
  - Then move up layer-by-layer.



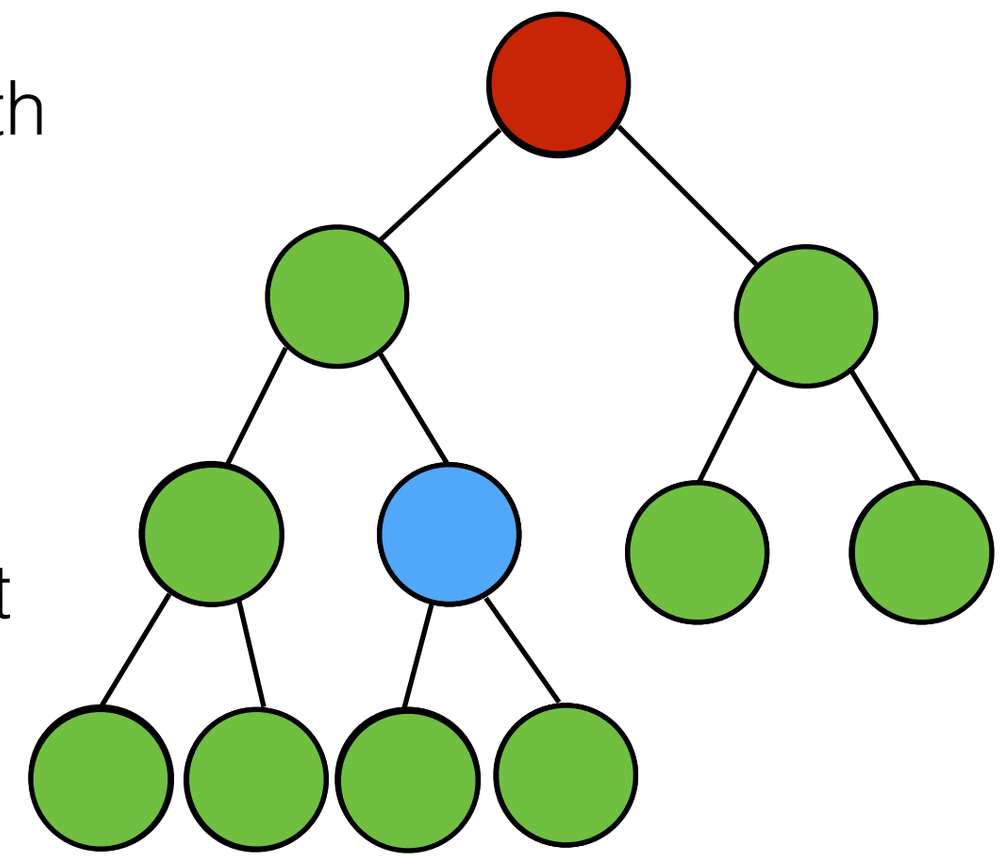
Heapify

- Start with an unordered array.
- `percolateDown(i)` assumes that both subtrees under *i* are already heaps.
- Idea: restore heap property bottom-up.
  - Make sure all subtrees in the two last layers are heaps.
  - Then move up layer-by-layer.



Heapify

- Start with an unordered array.
- `percolateDown(i)` assumes that both subtrees under *i* are already heaps.
- Idea: restore heap property bottom-up.
  - Make sure all subtrees in the two last layers are heaps.
  - Then move up layer-by-layer.

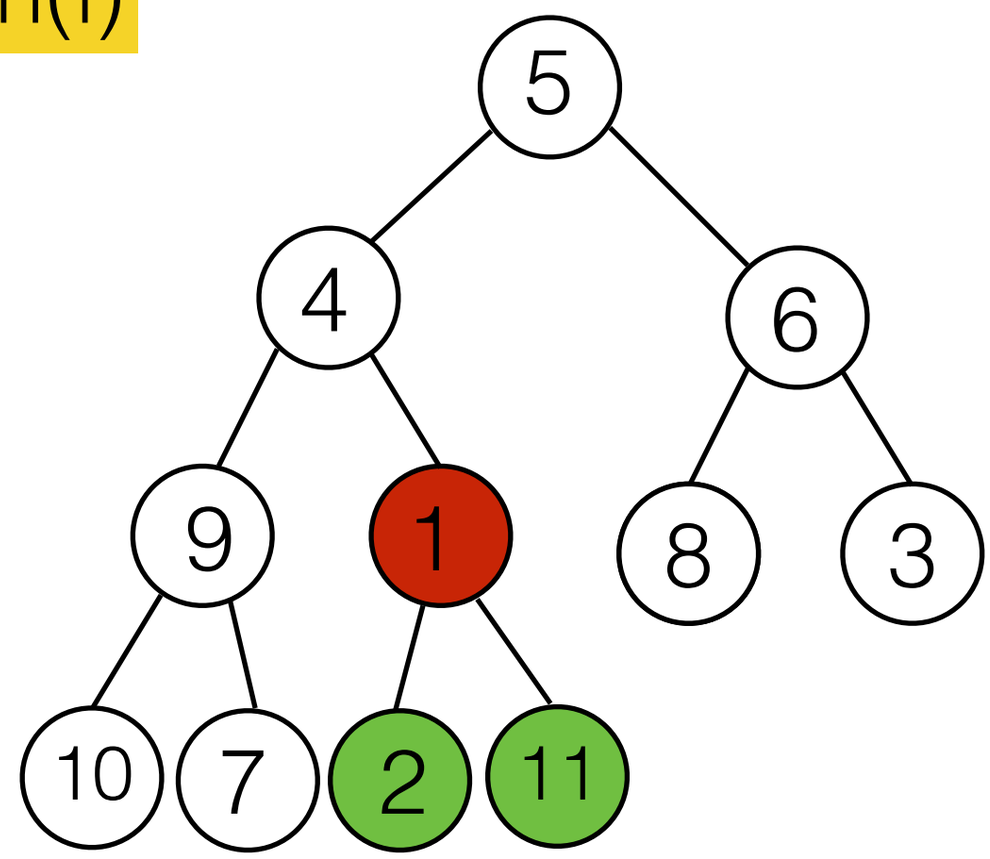


For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=11/2 = 5

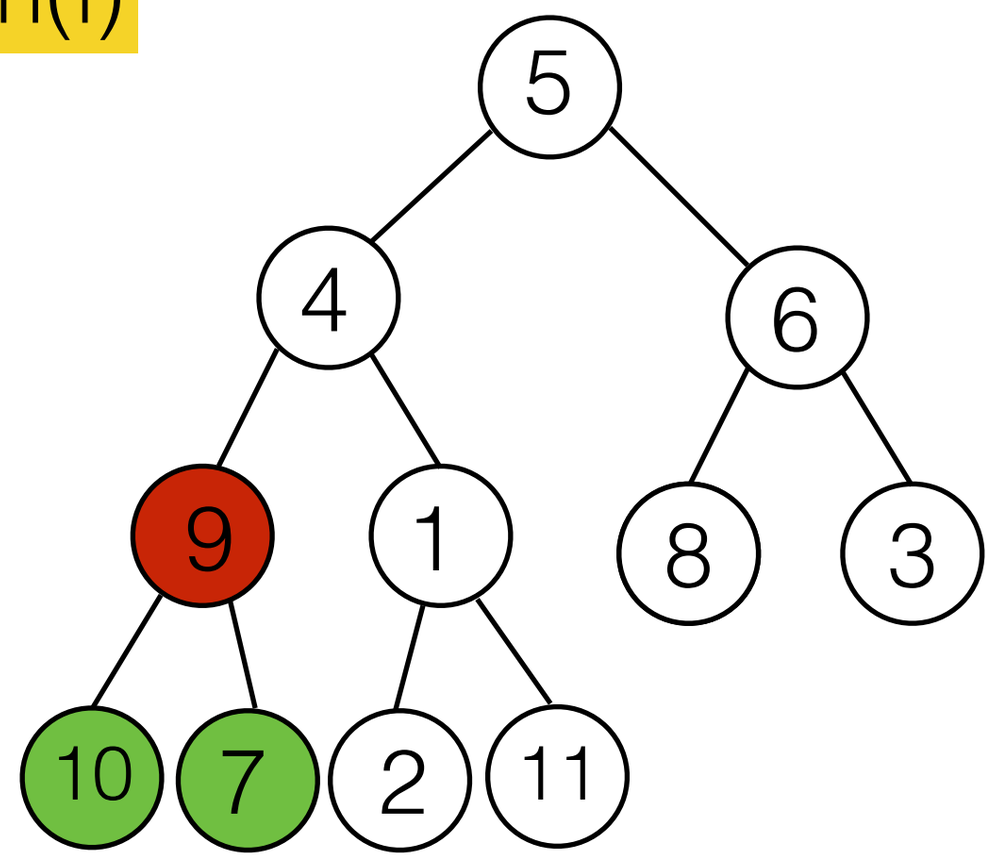


	5	4	6	9	1	8	3	10	7	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=4

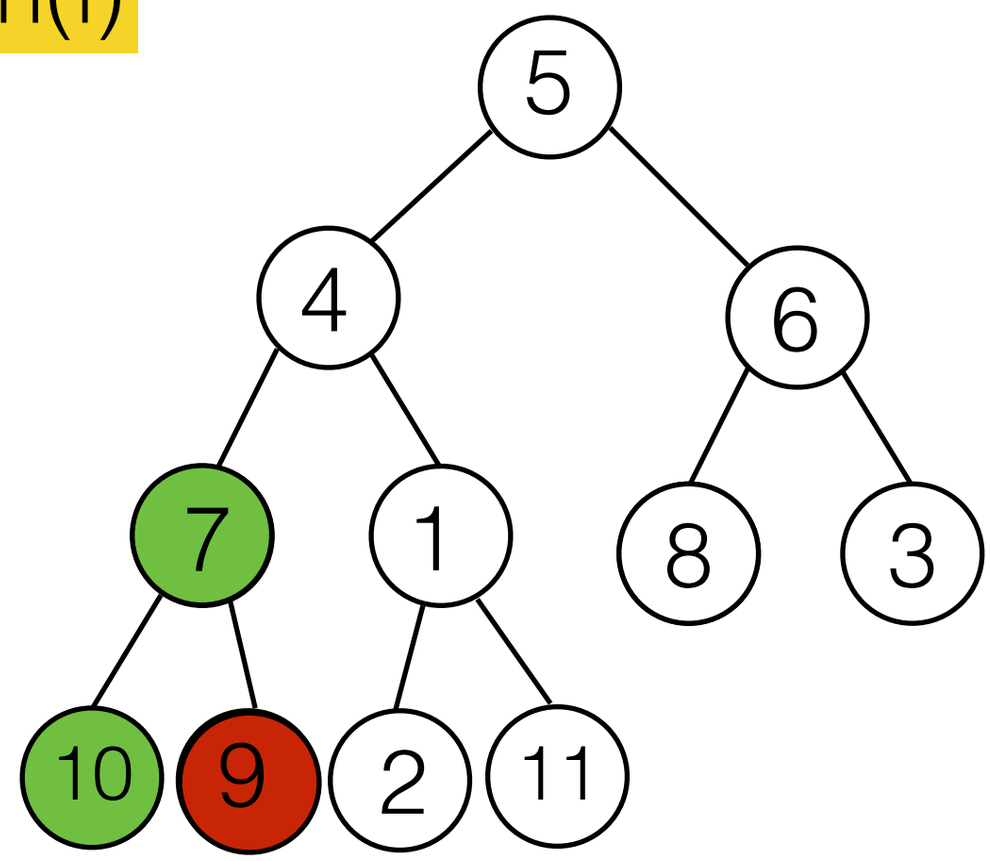


	5	4	6	9	1	8	3	10	7	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=4

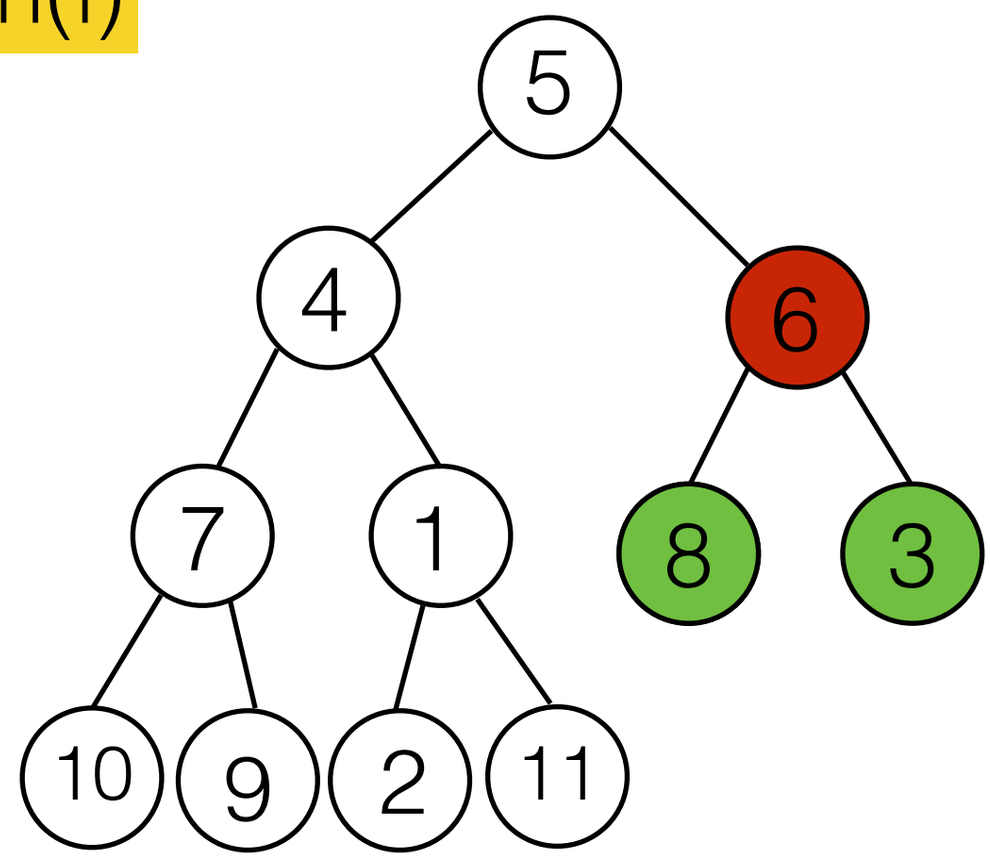


	5	4	6	7	1	8	3	10	9	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=3

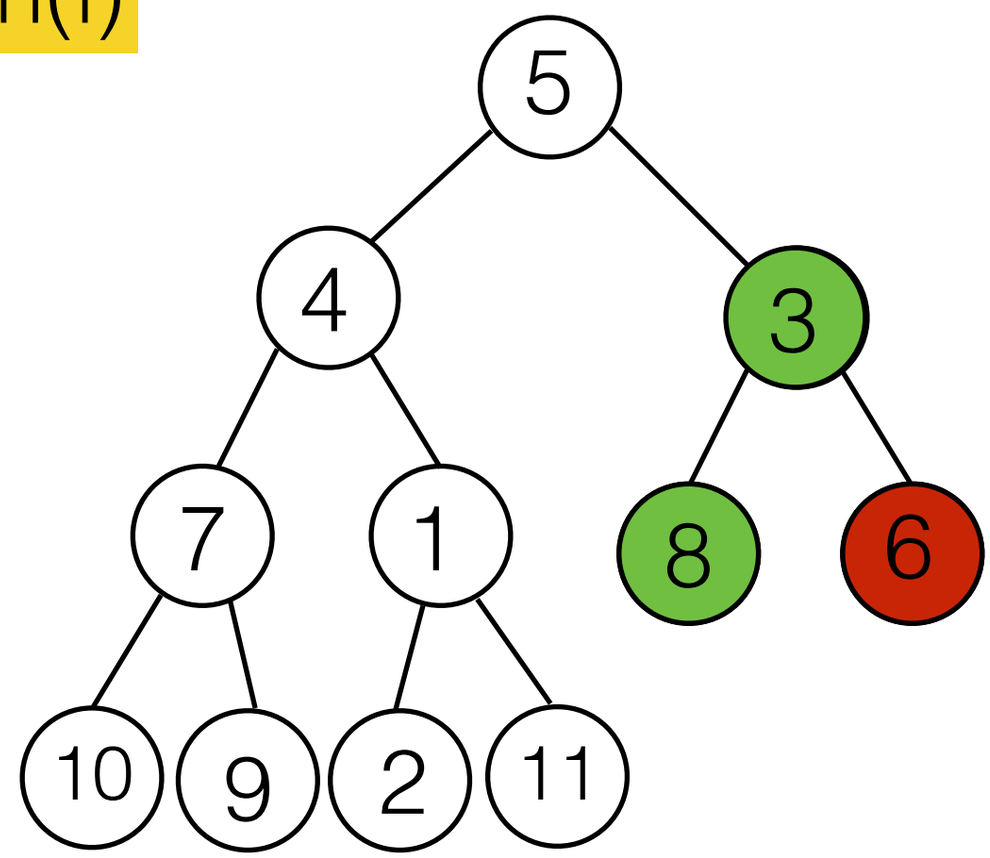


	5	4	6	7	1	8	3	10	9	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=3

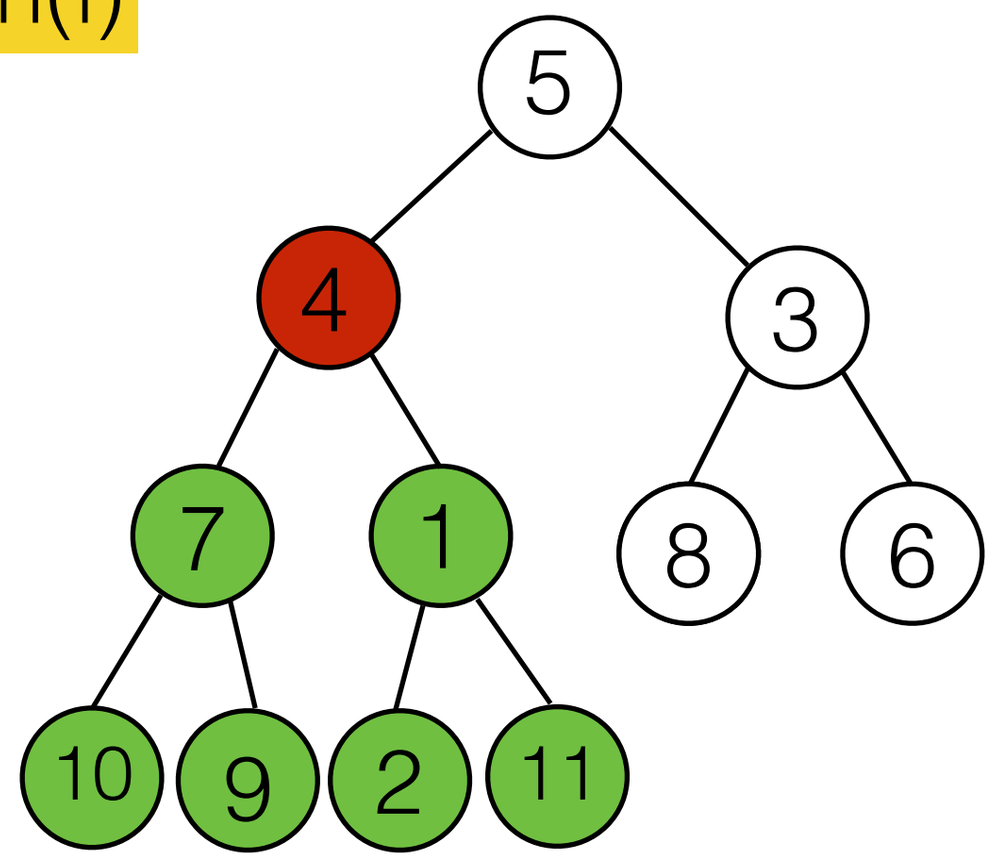


	5	4	3	7	1	8	6	10	9	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=2

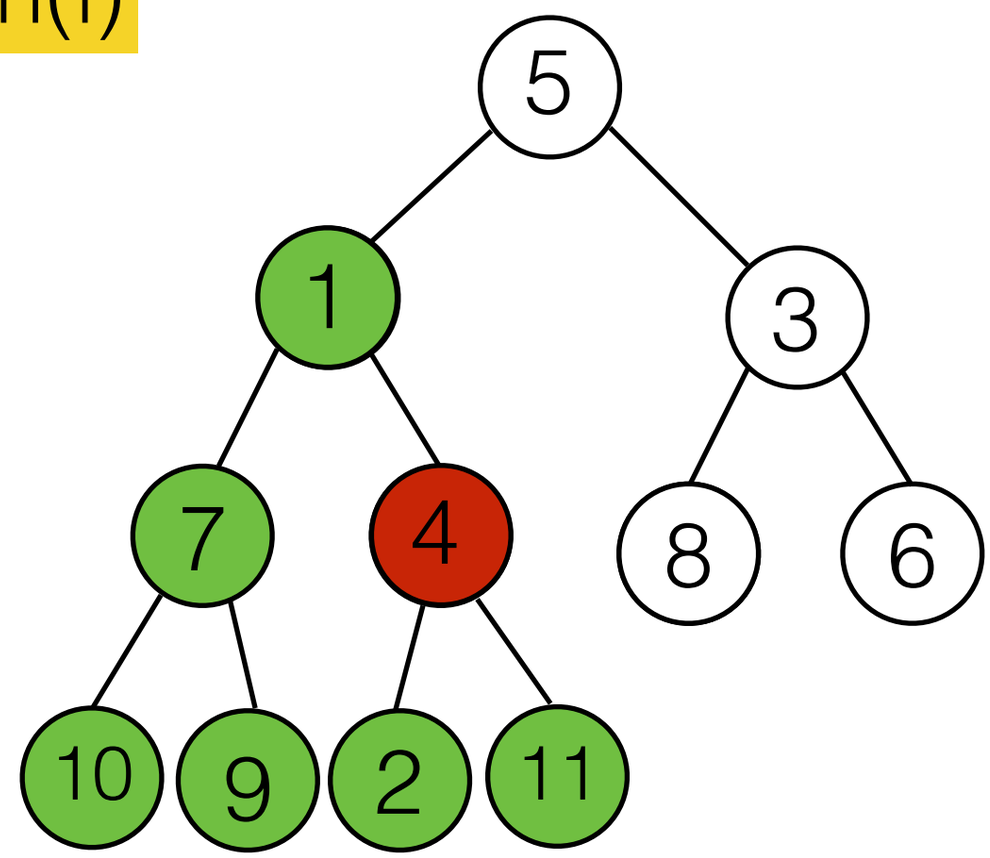


	5	4	3	7	1	8	6	10	9	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=2

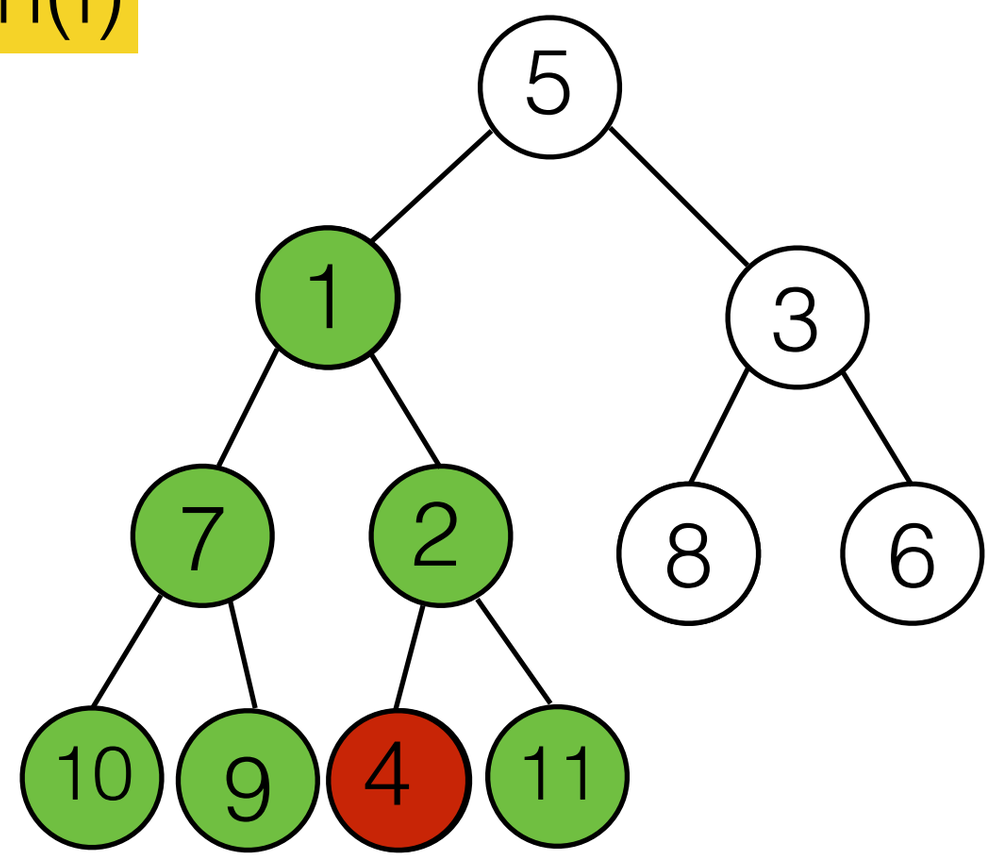


	5	1	3	7	4	8	6	10	9	2	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=2

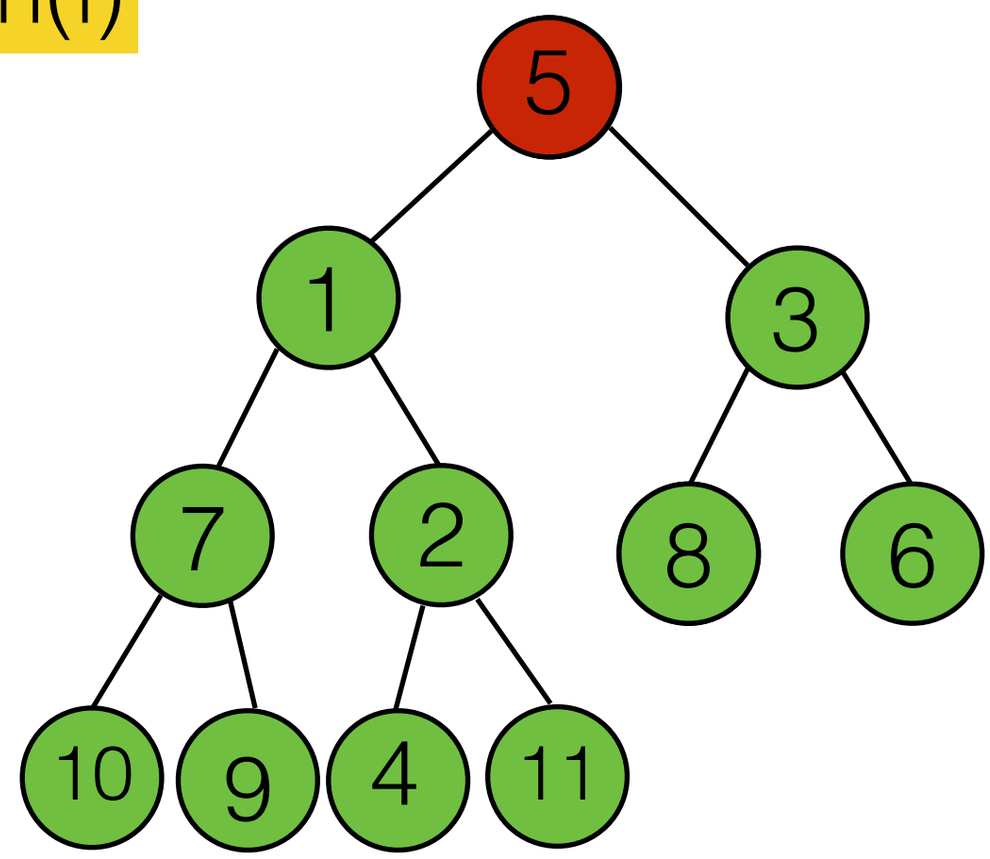


	5	1	3	7	2	8	6	10	9	4	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=1

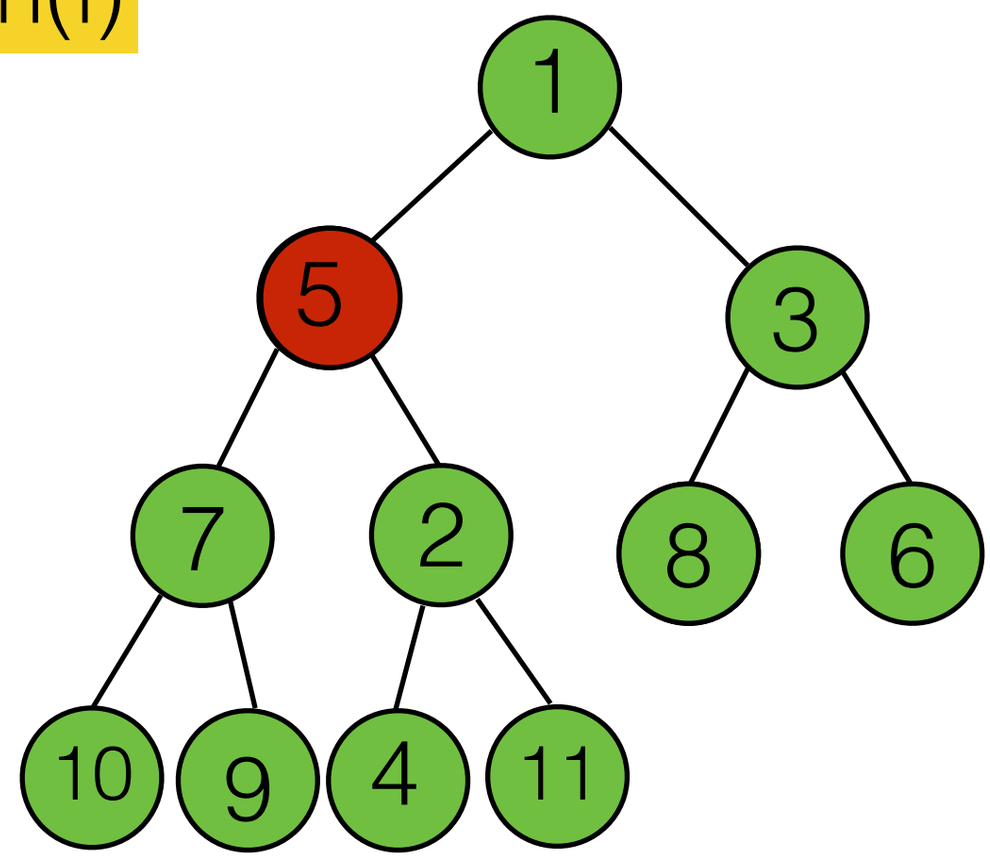


	5	1	3	7	2	8	6	10	9	4	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=1

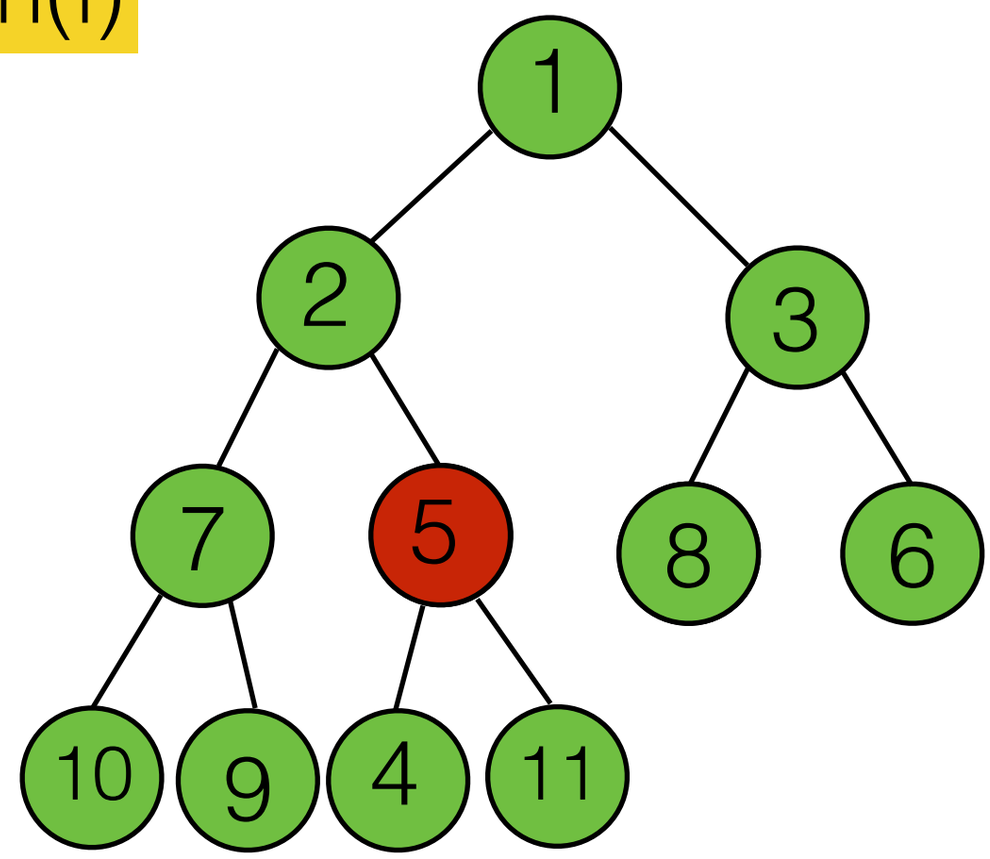


	1	5	3	7	2	8	6	10	9	4	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=1

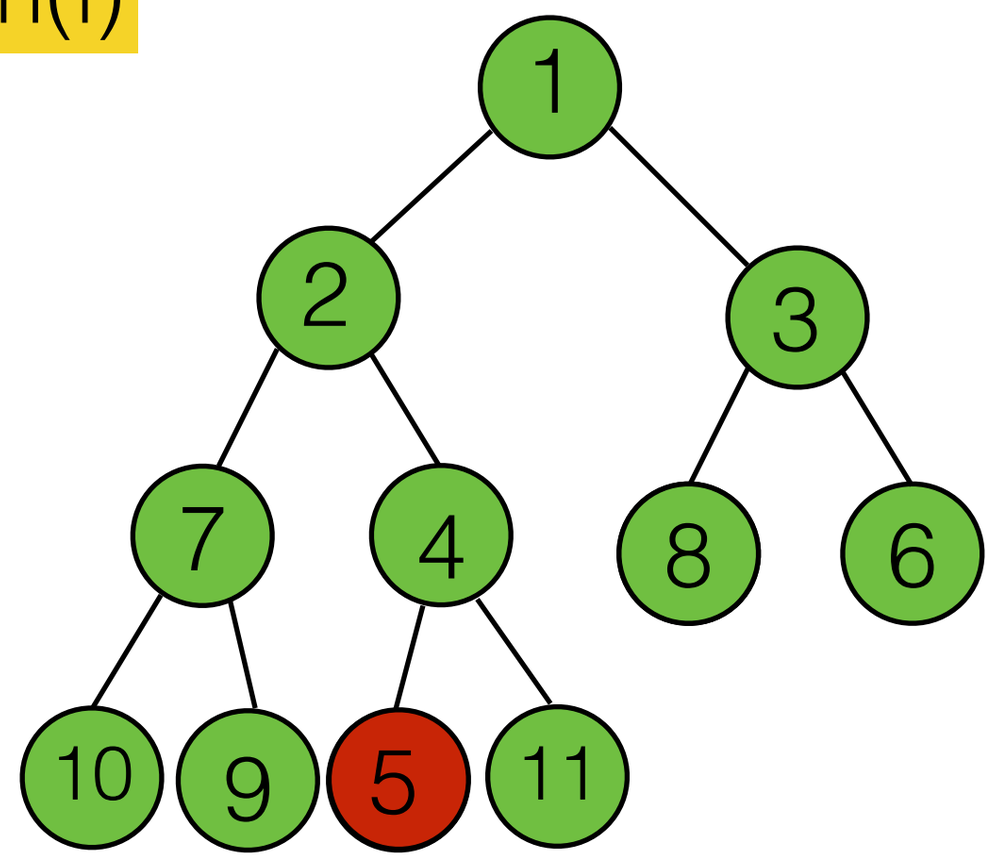


	1	2	3	7	5	8	6	10	9	4	11		

Heapify - Example

For i = $\lfloor N/2 \rfloor$ … 1
    percolateDown(i)

i=1



	1	2	3	7	4	8	6	10	9	5	11		

Heapify - Running Time

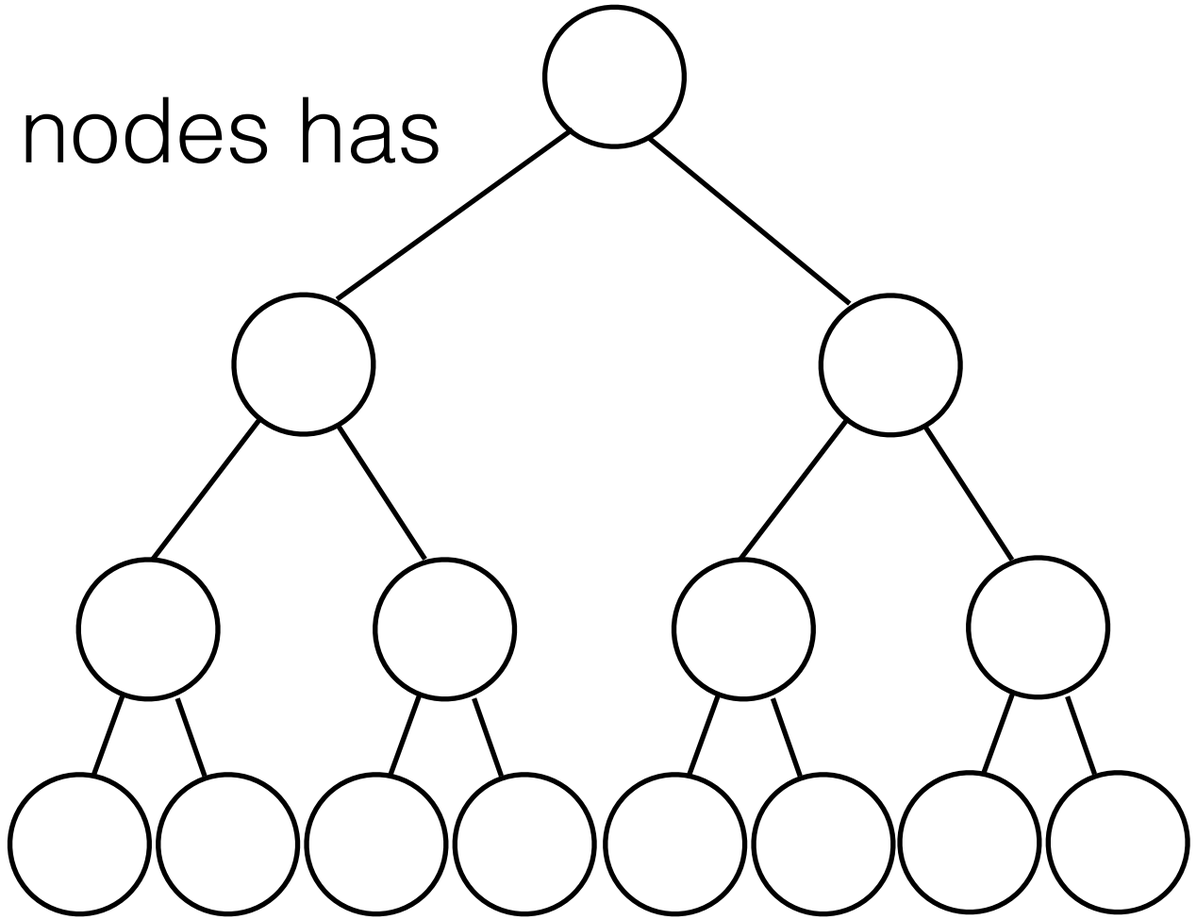
- How many comparisons do we need in each of the $N/2$ `percolateDown` calls?
  - In the worst case, each call to `percolateDown` needs to move the value all the way down to the leaf level.
  - We need to sum the possible steps for each level of the tree.

`BuildHeap` - Running Time

- Upper bound for nodes in a complete binary tree (if all levels are full = perfect binary tree) : $2^{h+1} - 1$

- A complete binary tree with N nodes has height: $h = \lfloor \log(N+1) \rfloor$

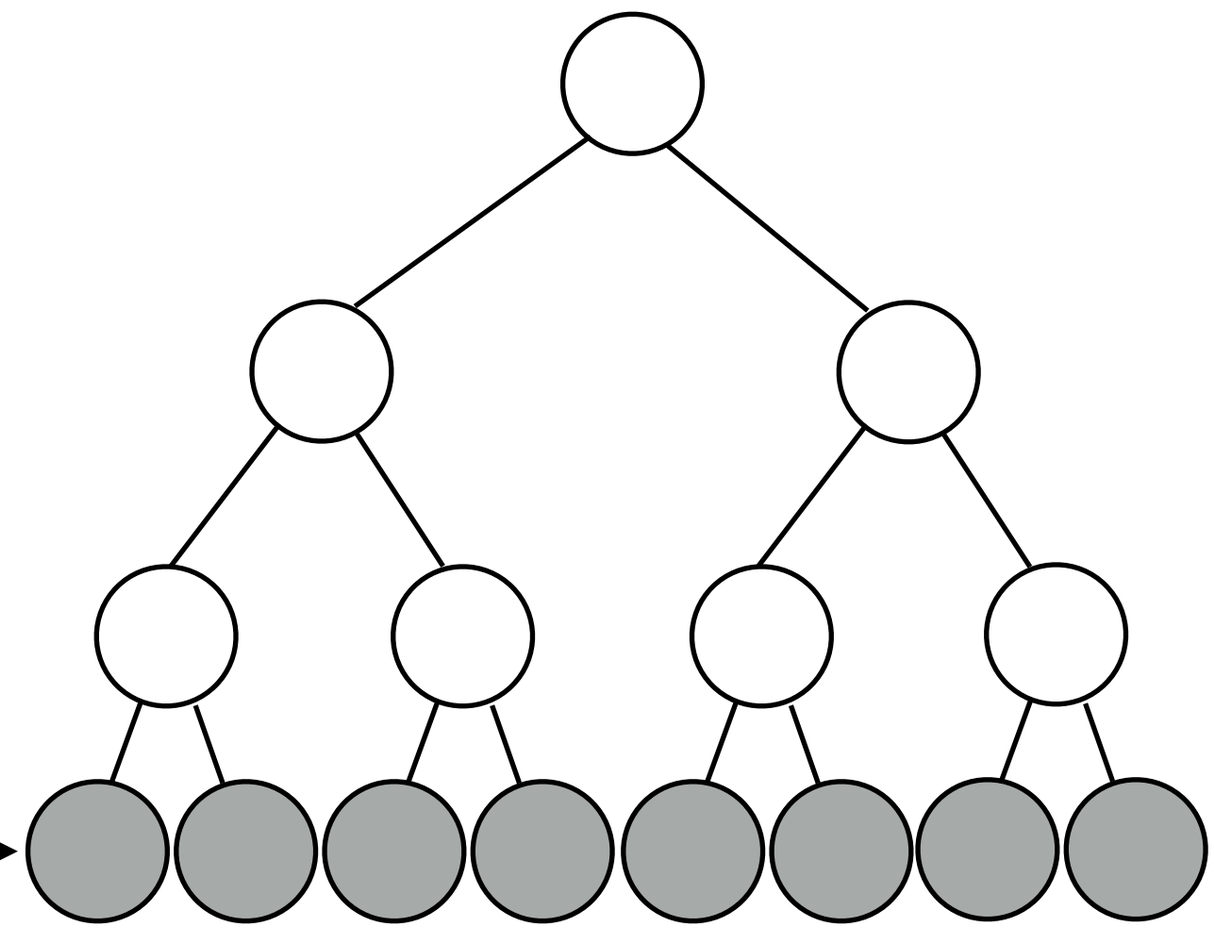
- In a complete binary tree of height h, there are 2^h leaves.



`BuildHeap` - Running Time

2^h nodes · 0 steps

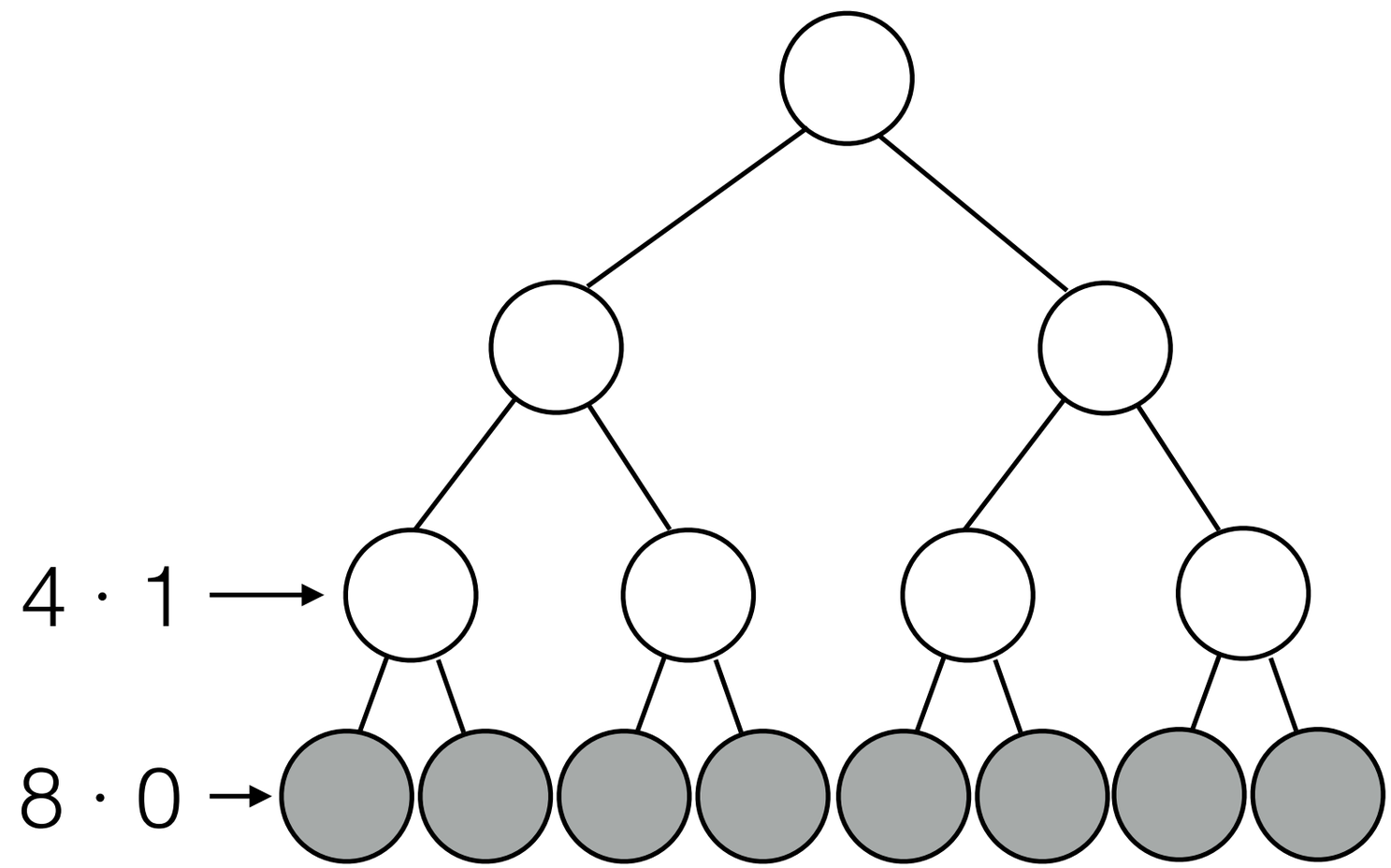
8 · 0 →



`BuildHeap` - Running Time

2^{h-1} nodes · 1 steps

2^{h} nodes · 0 steps

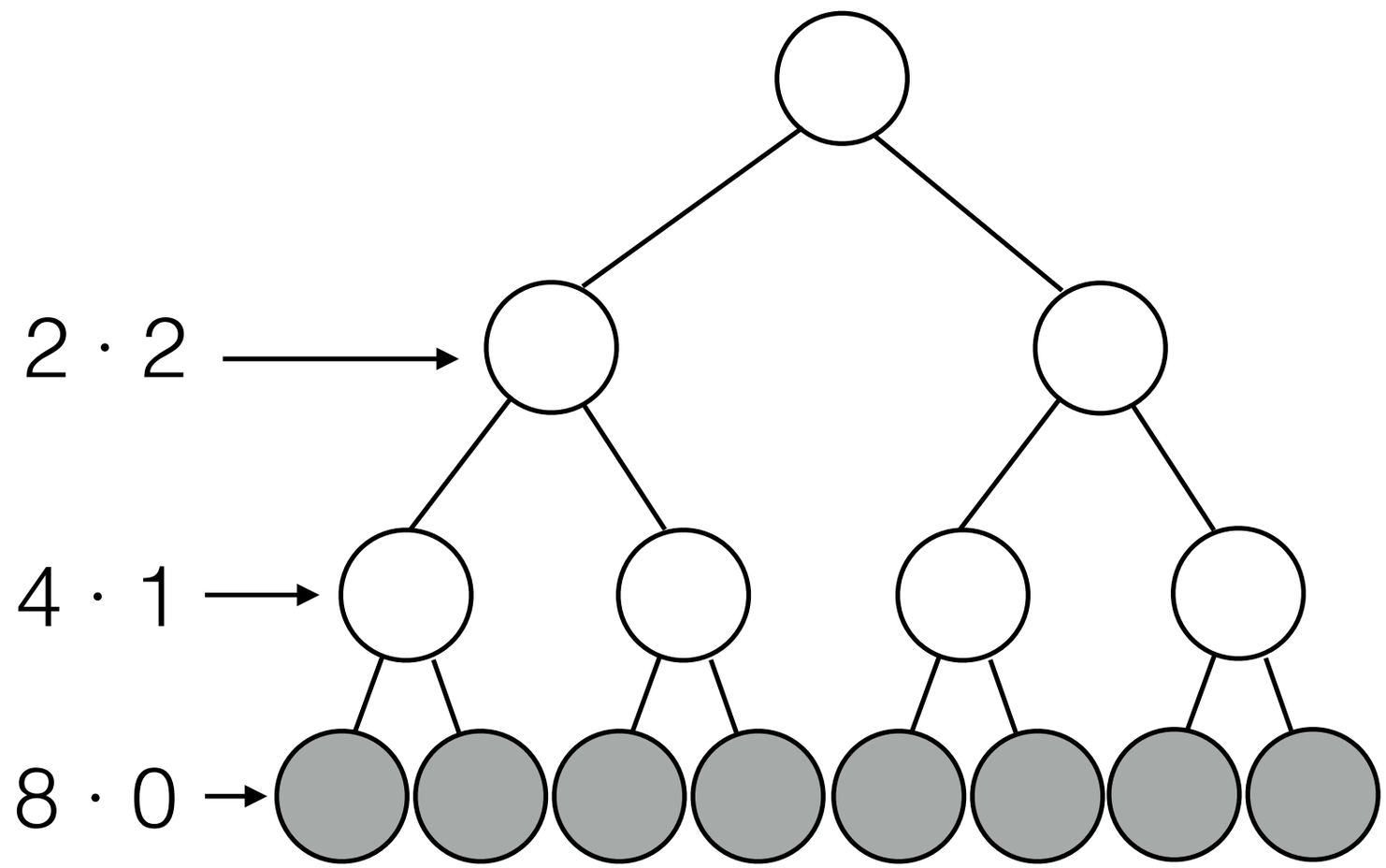


`BuildHeap` - Running Time

2^{h-2} nodes · 2 steps

2^{h-1} nodes · 1 steps

2^{h} nodes · 0 steps



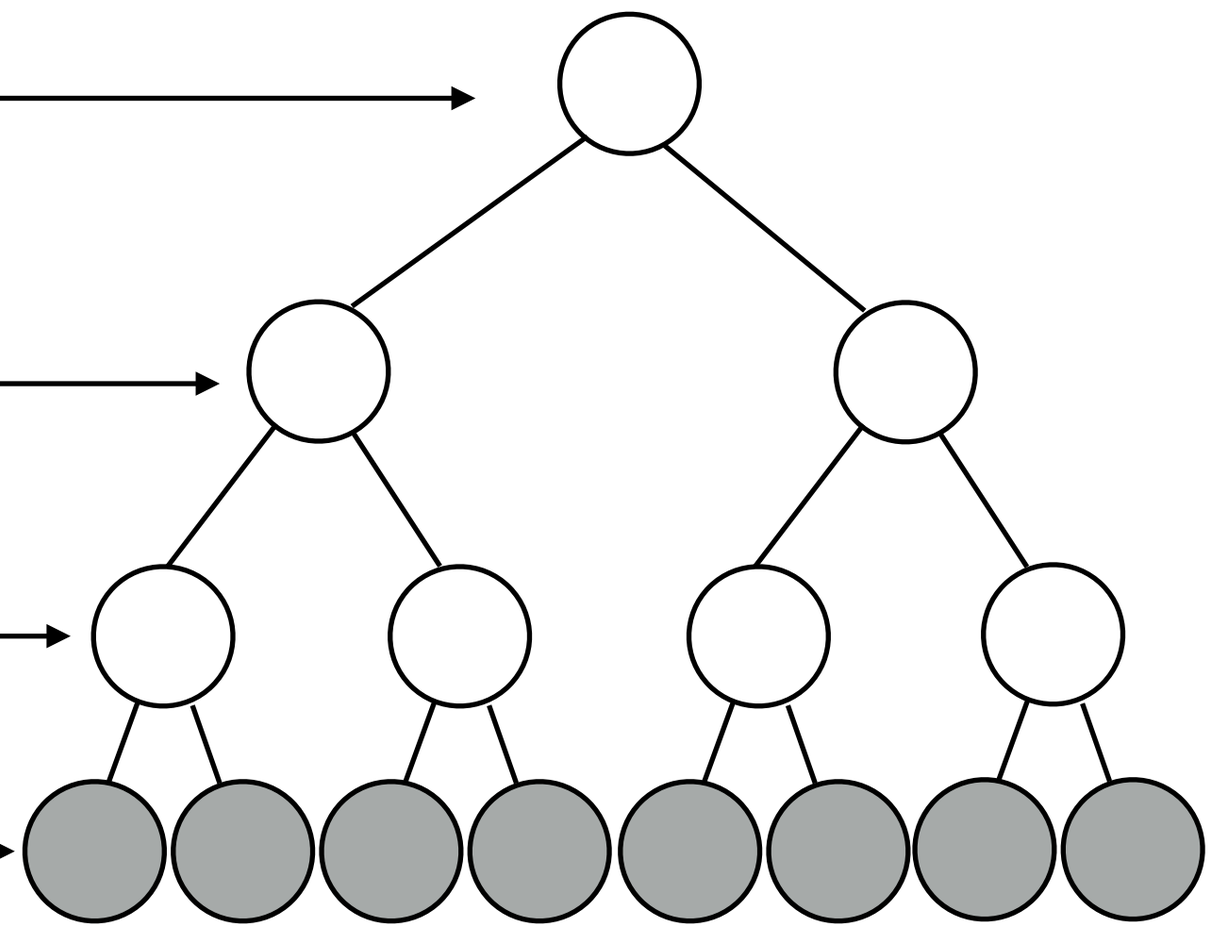
`BuildHeap` - Running Time

2^{h-3} nodes · 3 steps 1 · 3

2^{h-2} nodes · 2 steps 2 · 2

2^{h-1} nodes · 1 steps 4 · 1

2^{h} nodes · 0 steps 8 · 0



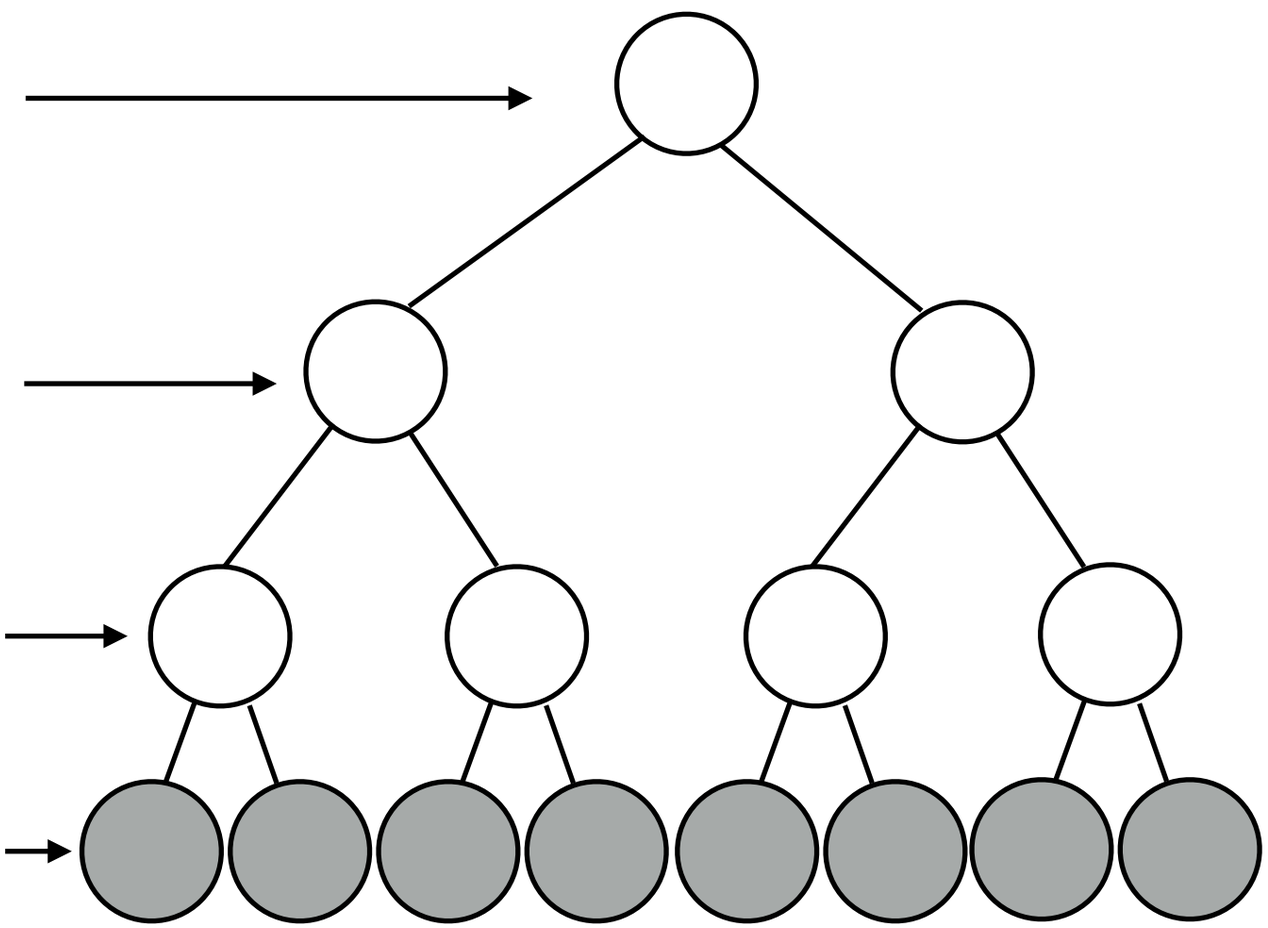
`BuildHeap` - Running Time

2^{h-3} nodes · 3 steps 1 · 3

2^{h-2} nodes · 2 steps 2 · 2

2^{h-1} nodes · 1 steps 4 · 1

2^{h} nodes · 0 steps 8 · 0



$$T(N) = 2^{h-1} \cdot 1 + \cdots + 4 \cdot (h-2) + 2 \cdot (h-1) + h \cdot 1$$

`BuildHeap` - Running Time

$$T(N) = 2^{h-1} \cdot (1) + 2^{h-2} \cdot (2) + \cdots + 4(h-2) + 2(h-1) + 1h$$

`BuildHeap` - Running Time

$$2T(N) = 2^h \cdot (1) + 2^{h-1} \cdot (2) + \cdots + 8(h-2) + 4(h-1) + 2h$$

$$T(N) = 2^{h-1} \cdot (1) + 2^{h-2} \cdot (2) + \cdots + 4(h-2) + 2(h-1) + 1h$$

`BuildHeap` - Running Time

$$2T(N) = 2^h \cdot (1) + 2^{h-1} \cdot (2) + \cdots + 8(h-2) + 4(h-1) + 2h$$

$$T(N) = 2^{h-1} \cdot (1) + 2^{h-2} \cdot (2) + \cdots + 4(h-2) + 2(h-1) + 1h$$

$$2 \cdot T(N) - T(N) = 2^h + 2^{h-1} + \cdots + 8 + 4 + 2 - h$$

`BuildHeap` - Running Time

$$2T(N) = 2^h \cdot (1) + 2^{h-1} \cdot (2) + \cdots + 8(h-2) + 4(h-1) + 2h$$

$$T(N) = 2^{h-1} \cdot (1) + 2^{h-2} \cdot (2) + \cdots + 4(h-2) + 2(h-1) + 1h$$

$$2 \cdot T(N) - T(N) = \underbrace{2^h + 2^{h-1} + \cdots + 8 + 4 + 2}_{(\sum_{i=0}^{h} 2^i) - 1 = (2^{h+1} - 1) - 1} - h$$

`BuildHeap` - Running Time

$$2T(N) = 2^h \cdot (1) + 2^{h-1} \cdot (2) + \cdots + 8(h-2) + 4(h-1) + 2h$$

$$T(N) = 2^{h-1} \cdot (1) + 2^{h-2} \cdot (2) + \cdots + 4(h-2) + 2(h-1) + 1h$$

$$2 \cdot T(N) - T(N) = \underbrace{2^h + 2^{h-1} + \cdots + 8 + 4 + 2}_{(\sum_{i=0}^{h} 2^i) - 1 = (2^{h+1} - 1) - 1} - h$$

$$T(N) = (2^{h+1} - 1) - (h + 1)$$

$$T(N) = (2^{h+1} - 1) - (\log(N + 1) + 1)$$

=N

`BuildHeap` - Running Time

$$2T(N) = 2^h \cdot (1) + 2^{h-1} \cdot (2) + \cdots + 8(h-2) + 4(h-1) + 2h$$

$$T(N) = 2^{h-1} \cdot (1) + 2^{h-2} \cdot (2) + \cdots + 4(h-2) + 2(h-1) + 1h$$

$$2 \cdot T(N) - T(N) = \underbrace{2^h + 2^{h-1} + \cdots + 8 + 4 + 2}_{(\sum_{i=0}^{h} 2^i) - 1 = (2^{h+1} - 1) - 1} - h$$

$$T(N) = (2^{h+1} - 1) - (h+1)$$

$$T(N) = (2^{h+1} - 1) - (\log(N+1) + 1) \quad = O(N)$$

=N

Heaps in Python

```
>>> import heapq as hq
>>> heap = []
>>> hq.heappush(heap, 4)
>>> hq.heappush(heap, 1)
>>> hq.heappush(heap, 2)
>>> hq.heappush(heap, 5)
>>> heap
[1, 4, 2, 5]
>>> hq.heapop(heap)
1
>>> heap
[2, 4, 5]
```

Heaps in Python

- Python implements binary heap operations in the **`heapq`** module
- Provides functions that operate on lists containing the heap data

```
>>> import heapq as hq
>>> heap = []
>>> hq.heappush(heap, 4)
>>> hq.heappush(heap, 1)
>>> hq.heappush(heap, 2)
>>> hq.heappush(heap, 5)
>>> heap
[1, 4, 2, 5]
>>> hq.heapop(heap)
1
>>> heap
[2, 4, 5]
```